



GEMLIDS-MIOT: A Green Effective Machine Learning Intrusion Detection System based on Federated Learning for Medical IoT network security hardening[☆]

Iacovos Ioannou^{d,e,*}, Prabagarane Nagaradjane^a, Pelin Angin^b, Palaniappan Balasubramanian^c, Karthick Jeyagopal Kavitha^f, Palani Murugan^g, Vasos Vassiliou^{d,e}

^a Department of ECE, Sri Sivasubramaniya Nadar College of Engineering, Chennai, India

^b Department of Computer Engineering at Middle East Technical University, Ankara, Turkey

^c School of Electronics and Computer Science, Bethnal Green, London E1 NS, United Kingdom

^d Department of Computer Science University of Cyprus, Nicosia, Cyprus

^e CYENS Center of Excellence, Nicosia, Cyprus

^f Groww Ltd, Bengaluru, India

^g Global Analytics India Pvt. Ltd, Chennai, India

ARTICLE INFO

Keywords:

Brute force authentication
DDoS
Green computing
Intrusion detection
MIoT
IoMT
Machine learning
MQTT
MitM
Raspberry Pi
Federated learning

ABSTRACT

The increasing use of Internet of Things (IoT) gadgets in a daily rate has heightened security apprehension, particularly within the healthcare sector. In order to prevent the unauthorized disclosure of sensitive data, it is imperative for Internet of Things (IoT) systems to promptly and effectively respond to harmful activities. Nevertheless, the act of transferring data to distant cloud servers for the purpose of analysis gives rise to both temporal delays and apprehensions regarding privacy. To ensure the security of medical Internet of Things (MIoT) networks, a power-efficient Intrusion Detection System (IDS) is employed for three primary objectives that it will result in three stages of execution: (i) The objective is to categorize different types of attacks, such as Man-in-the-Middle (MitM) and Distributed Denial of Service (DDoS), by utilizing well-established machine learning (ML) techniques. This classification stage will serve to enhance the Intrusion Detection System (IDS) and the reporting system. (ii) Anomaly detection (unknown attack identification), or detection of unknown attacks, will be employed to identify previously unknown attacks. This identification stage involves retraining the ML model to enable future recognition and classification of these unknown attacks when the anomaly attack detector identifies that an unknown attack is recognized. Then, a retraining of the first stage classification model is executed due to the anomaly detection. (iii) To ensure that a remote cloud server remains current with the latest classification model changes, Federated Learning (FL) will be utilized. FL allows for collaborative model training while preserving data privacy and security. The experimental findings indicate that the Enhanced Random Forest (also called ensemble random forest) algorithm achieves a remarkable accuracy rate of 99.98% in classifying attacks. Thus, it will be our first stage classifier. Continuing, the One-Class Support Vector Machine (SVM) algorithm demonstrates a high level of accuracy, reaching 99.7% in detecting anomalies so that it will be our second stage identifier. Finally, the third-stage approach, which has as a target the overall system model updater, will be our introduced Federated Learning approach that works with the Enhanced Random Forests and identifies the ERF differences from the old model in an optimal way. The efficacy of our technique is confirmed through the implementation of experiments involving an Internet of Things (IoT) system and a Raspberry Pi MIoT gateway and with simulations that simulate the FL updating process. These experiments successfully identify known and unknown attacks with a high reliability level while limiting resource utilization and energy consumption. Future studies of this work will focus on enhancing the scalability and efficiency of our Intrusion Detection System in MIoT networks.

[☆] Acknowledgments: The authors would like to thank the technical staff at the Department of Computer Science, University of Cyprus and CYENS Center of Excellence for their support and assistance during this research.

* Corresponding author at: Department of Computer Science University of Cyprus, Nicosia, Cyprus.

E-mail address: ioann06@cs.ucy.ac.cy (I. Ioannou).

<https://doi.org/10.1016/j.comcom.2024.02.023>

Received 23 September 2023; Received in revised form 31 December 2023; Accepted 22 February 2024

Available online 24 February 2024

0140-3664/© 2024 Elsevier B.V. All rights reserved.

1. Introduction

The healthcare sector has witnessed a significant increase in security concerns about handling sensitive data, primarily due to the widespread adoption of Internet of Things (IoT) devices. To protect sensitive data within the network, it is imperative for these devices to promptly and effectively respond to any hostile activity. The challenges that emerge in the realm of Internet of Things (IoT) applications stem from their heterogeneous character, the absence of comprehensive security solutions at the outset, and the manufacturers' limited focus on security. In addition, the constrained computational capabilities of numerous Internet of Things (IoT) devices provide obstacles to the effective implementation of security software. Despite implementing IoT-enabling technology and intrusion prevention systems, attackers continue to compromise devices successfully. The identified vulnerabilities have noteworthy ramifications for medical IoT (MIoT) or Internet of Medical Things (IoMT) networks that transmit critical patient data. In order to ensure the security of IoT networks, we suggest the implementation of an Intrusion Detection System (IDS) to actively monitor MIoT gateways, also known as sinkholes, by employing low-power Machine Learning (ML) techniques.¹ This study presents an innovative methodology that centers on environmentally friendly technology, employing various machine learning approaches to categorize numerous network assaults on MIoT gateways/sinkholes and detect unfamiliar attacks as abnormal/anomalous occurrences. Furthermore, we present a methodology that utilizes Federated Learning (FL) to effectively update other Mobile Internet of Things (MIoT) gateways through a cloud server.

Regarding employing environmentally friendly technology, our research on MIoT network security incorporates a power-efficient Intrusion Detection System (IDS). This system's design focuses on resource optimization, which reduces energy consumption and aligns with environmental sustainability goals. Our methodology unfolds in three stages: (1) The first stage involves using Enhanced Random Forest algorithms for accurate attack classification. (2) the One-Class Support Vector Machine (SVM) efficiently detects anomalies in the second stage. (3) The third stage employs Federated Learning (FL) to collaboratively update the model while ensuring data privacy and reducing the need for extensive data transmission. The pivot of our study towards energy efficiency, especially in the application of FL, aims to minimize the energy requirements of MIoT devices. These devices often have strict power constraints, and our approach to optimizing memory, CPU, and disk space usage significantly reduces their energy consumption. Our experiments, which include real Medical IoT systems and simulations, have demonstrated our system's capability to detect known and unknown attacks regarding resource usage reliably and conservatively. This approach not only enhances the security of MIoT networks but also aligns with our commitment to developing environmentally sustainable solutions in the realm of healthcare cybersecurity.

Prioritizing green energy sources is of utmost importance, mainly because sinkholes rely on batteries as integral components of their Internet of Things (IoT) functionality. Hence, the power consumption of machine learning techniques becomes a critical factor to consider while implementing an Intrusion Detection System (IDS) on a system with limited resources. The energy conservation for MIoT devices can be achieved by deploying the Intrusion Detection System (IDS) on the sinkhole instead of installing it on each individual device inside the network. The sinkhole/gateway can also function as an access point, enabling sharing of its bandwidth through Wi-Fi Direct. Furthermore, its secondary interface operates in monitor mode, thereby enhancing data security and minimizing latency. Moreover, it is worth noting that

all sinkholes inside the MIoT network undergo training upgrades to improve the identification of attacks by utilizing Transfer/Federated Learning techniques.

Our study offers a thorough and all-encompassing approach to improving security in the medical Internet of Things (MIoT). This is achieved by utilizing a unique combination of modern machine-learning methods within a unified framework. Our approach primarily utilizes a sophisticated combination of a one-class Support Vector Machine (SVM) to accurately identify extremities in MIoT networks and Enhanced Random Forests to detect a wide range of threats. This fusion greatly enhances the precision and effectiveness of detecting and categorizing network vulnerabilities. Our novel approach relies on utilizing our novel Federated Learning approach that uses Enhanced Random Forests to enable dynamic and continuous upgrades of the Intrusion Detection System (IDS). This method allows for quickly and effectively incorporating new attack patterns into the MIoT network. It utilizes specialized techniques for traversing trees to ensure the proper distribution of models. Our overall proposed approach consists of three key steps: first, using sophisticated classification techniques at MIoT gateways to identify network threats, known and unknown; second, incorporating the unknown identified system attacks into the IDS by training the classification model with these new attack records; and third, continuously updating both the remote cloud server and the MIoT gateways using our introduced Federated Learning approach that calculates the weights of enhanced random forest in an optimal way. The Federated Learning approach utilizes an Enhanced Random Forest machine learning model in the cloud to update the MIoTs in the Enterprise Resource Planning (ERP) system. The implementation of this three-step execution strategy brings about a significant change in conventional IDS methodologies, offering a security solution that is capable of adapting in real-time and delivering accurate protection for MIoT environments. The functionality of our system is improved by using cloud-based transfer learning, enabling the IDS to be optimized in real-time. This functionality enhances the accuracy of models for one or all MIoT sinkholes, adjusting to the constantly changing network threat environment. The effectiveness of our all-inclusive solution is convincingly proved by rigorous testing in a specifically engineered emulation environment, verifying its capacity to proficiently and precisely identify a broad spectrum of threats. Moreover, this comprehensive solution with the three techniques (i.e., Enhanced Random Forests, one-class SVM, and FD at Random Forests) effectively addresses the crucial resource allocation problem in MIoT contexts, attaining an ideal equilibrium in power, CPU utilization, and memory. The bilateral model update process between the cloud and MIoT components guarantees that all network elements stay synced with the latest threat detection models. Our paper presents a comprehensive and innovative solution for enhancing security in MIoT (Mobile Internet of Things) networks. This solution integrates advanced machine learning techniques with a novel system update strategy, resulting in an adaptive Intrusion Detection System (IDS) that effectively addresses the specific challenges and requirements of MIoT networks.

The primary contributions of this research work can be briefly summarized as follows:

- We propose an energy-efficient IDS approach for MIoT networks, which runs lightweight ML models at the sinkhole for real-time detection of attacks.
- We classify the MitM, DDoS, Brute Force Authentication, and NMAP attacks for securing our MIoT network using Enhanced Random Forests.
- We identify unknown attacks for securing our MIoT network using One-Class SVM as anomalies.
- We propose a cyber threat intelligence architecture for MIoT, where newly discovered attack data are shared through the cloud to update the ML models at the connected gateways. The proposed attack data-sharing model preserves the privacy of sensitive data.

¹ We have optimized the system to minimize resource use—including memory, CPU, and disk space—thereby reducing the energy requirements of MIoT devices.

- We demonstrate through experiments in an MIIoT environment using Raspberry Pi and connected sensors that the proposed approach with the new dataset achieves high accuracy, low power consumption, and low detection time, compliant with the requirements of MIIoT applications.
- We introduce an FL at the MIIoT network, which will have the network's sinkhole(s) updated with the recent changes of nodes and trees in their Enhanced Random Forest model by the cloud server for additional attack identification. Furthermore, we modified the FL to be used with Enhanced Random Forests and MIIoT devices.
- We show that the proposed approach that encompasses Enhanced Random Forest, and One-Class SVM is best among other approaches regarding the highest accuracy, lowest power consumption, CPU utilization, and memory.

The rest of the paper is structured as follows. An overview of related work in the field of MIIoT intrusion detection is provided in Section 2.1. Section 2.2 includes background information on IoT and intrusion detection. The assumptions, terms, system model, and problem description are provided in Section 3. Section 4 describes the proposed end-to-end intrusion detection approach and federated learning. The used ML models are briefly analyzed and described in Section 4.4. The investigated approaches' performances are examined, evaluated, and compared in Section 5. Finally, Section 6 includes concluding remarks and our future work directions.

2. Related work and background information

This section provides the related work to our investigation and the background information related to our work. In MIIoT, there is no work associated with Federated Learning; our approach is the only one supporting Federated Learning. Moreover, no method exists that jointly identifies unknown attacks and then classifies these attacks in a power and memory-save manner.

2.1. Related work

This section provides the relevant work related to MIIoT and our investigation. More specifically, due to the detrimental effects of cyber-attacks on IoT systems, particularly those used in healthcare, intrusion detection in IoT has become a significant field of research in recent years. IDS for IoT, like IDS for legacy systems, can be called signature-based or anomaly-based. An anomaly-based IDS compares a system's behavior to a predefined profile of regular activity and generates alerts whenever an action deviates from routine behavior by exceeding a predefined threshold. As a result, it effectively detects and prevents new attacks (e.g., in the IoT context, abuse of resources). Because any deviation from established boundaries is interpreted as an attack, all acceptable behavior must be known. This is impractical, however, because normal behaviors change over time. As a result, this technique is prone to generating false positives. The expected behavior profiles that statistical methods or machine learning algorithms can cause may need to be more significant for IoT network nodes with limited space and resources. Various approaches for intrusion detection in MIIoT have been proposed in recent years.

2.1.1. IDS approaches for MIIoT

Authors in [1] presented an intrusion detection approach for Medical Internet of Things (MIIoT) networks. They employed machine learning techniques such as Decision Trees (DT), Support Vector Machines (SVM), and K-Means on MIIoT gateways. The research focused on utilizing simulator data for evaluation and did not specify particular attacks. Moreover, their work did not address power consumption or incorporate Federated Learning (FL) for model updates, which are vital aspects for resource-constrained MIIoT devices.

Authors in [2] proposed an intrusion detection system (IDS) for connected healthcare systems (CHS) using Autoencoders and XG-Boost. Unfortunately, details about the dataset they used were not publicly available. They primarily identified attacks like Interception, Forgery, and Tampering. However, like Gao et al. [1], this work did not consider power consumption analysis or FL for real-time model updates, which are essential in MIIoT environments.

In their work, Authors in [3] introduced an approach for cyber-attack detection in patient monitoring devices (PMD) using techniques like N-grams, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest (RF), and Decision Trees (DT). They utilized actual device data to identify attacks such as Eavesdropping, Denial of Service (DoS), Man-in-the-Middle (MitM), Replay, and False Data Injection. However, like prior studies, this research did not account for power consumption analysis or implement Federated Learning.

Authors in [4] designed a Mobile Agent-based intrusion detection system (IDS) for securing medical device networks. They utilized Machine Learning and regression algorithms to detect attacks like DoS, Data Falsification, and Passive Listening. Unfortunately, the dataset details were not publicly available, and the study did not incorporate power consumption analysis or Federated Learning into the solution.

Authors in [5] proposed an intrusion detection system for MIIoT using Principal Component Analysis (PCA), Grey Wolf Optimization (GWO), and Deep Neural Networks (DNN). They focused on attacks like Denial of Service (DoS), User-to-Root, Probe, and Remote to Local attacks using the Kaggle Intrusion dataset. However, the study did not consider power consumption analysis or Federated Learning.

Authors in [6] developed an intrusion detection system for MIIoT networks based on Naive Bayes (NB), Decision Trees (DT), Random Forest (RF), and XGBoost algorithms. They used the Ton-IoT dataset but did not specify particular attacks. Similar to previous works, this study did not address power consumption analysis or incorporate Federated Learning.

In their research, Authors in [7] focused on a healthcare Internet of Things (IoT) Intrusion Detection System (IDS) using Convolutional Neural Networks (CNN). They classified firewall risk levels as Normal, Critical, Major, and Minor. However, specific dataset details were not publicly available, and the work did not consider power consumption analysis or Federated Learning.

Authors in [8] designed an IDS for healthcare systems using Support Vector Machines (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), and Artificial Neural Networks (ANN). Using publicly available datasets, they identified attacks like MitM (Spoofing and Data Alteration). Nevertheless, the research did not include power consumption analysis and Federated Learning.

Authors in [9] introduced a framework for detecting attacks in Fog nodes using EOS-ELM. They utilized the NSL-KDD dataset to identify attacks like MitM and DDOS. However, the study did not involve power consumption analysis or Federated Learning.

In their work on MIIoT Malware Detection, Authors in [10] employed Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM). Unfortunately, specific dataset details were not mentioned, and the research did not address power consumption analysis or Federated Learning.

In the [11], authors introduce the AnoFed, a novel framework for anomaly detection in digital healthcare. It combines transformer-based Autoencoders (AE) and Variational Autoencoders (VAE) with Support Vector Data Description (SVDD) in a federated learning setting. This approach aims to enhance privacy, improve explainability, and support adaptive anomaly detection. The effectiveness of AnoFed is demonstrated through experiments using ECG data for anomaly detection, showing significant efficiency and effectiveness compared to state-of-the-art methods. More precisely, the study leverages transformer-based autoencoders and variational autoencoders in a federated learning setting integrated with Support Vector Data Description (SVDD) for adaptive anomaly detection. The framework, applied to ECG anomaly

detection, aims to improve privacy protection, enhance the explainability of results, and support adaptive detection.

2.1.2. IDS approaches for IoT

Many researchers have also proposed approaches for intrusion detection in general IoT systems. Authors in [12] adapted Suricata, a signature-based intrusion detection system (IDS), to detect denial-of-service attacks in 6LoWPAN networks. The system was developed to be deployed on a centralized, dedicated host. More specifically, the system checks IDS alerts for channel interference and the rate at which packets are dropped to confirm an attack and reduce false alarms.

In [13], a two-trained deep learning model-based anomaly detection system for the IIoT/IICSs is proposed. A Deep Auto Encoder (DAE) was initially learned during the training phase using only standard network behaviors to generate some parameters (e.g., weights and biases). The generated parameters are used as values to initialize a standard supervised Deep Feed Forward Neural Network (DFFNN) algorithm to identify existing and new attack instances during the testing phase. Two well-known network datasets were used to evaluate the model. Firstly, with the NSL-KDD dataset, the accuracy was 98.6%, while the false-positive rate was 1.8%. Secondly, with the UNSW-NB15 dataset, the accuracy was 92.4%, and the false-positive rate was 8.2%.

Also, in [14], the authors proposed a scheme based on a three-pattern detection algorithm that utilizes Snort and ClamAV intrusion pattern sets. The algorithm was evaluated on a Raspberry Pi equipped with an Omnivision 5647 sensor capable of capturing images for transmission to a central server. Even though the approach uses auxiliary shift values, the technique eliminates many unnecessary comparison operations between packet payloads and attack signatures, thereby lowering the computational cost on low-capacity IoT nodes. When resources were limited, the method was two times faster than the traditional pattern-matching algorithm.

The research in [15] proposed an architecture for detecting SYN flooding attacks in IoT networks that utilizes Random Neural Networks and LSTM. The authors generated their dataset by establishing a virtual network and capturing traffic in PCAP files.

A proposed sequential attack detection architecture for IoT networks that uses three machine learning models is found in [16].

For distributed attack detection in IoT networks [17], a fog-based semi-supervised learning approach was proposed. The authors demonstrated that their distributed approach outperformed centralized solutions regarding detection time and accuracy using the NSL-KDD dataset.

A botnet detection scheme based on anomalies in 6LoWPAN sensor networks is proposed in [18]. The solution monitors network traffic and notifies users when any node's computed averages undergo unexpected changes. The profile of acceptable behavior was constructed by averaging the TCP control field, packet length, and the number of connections for each sensor.

A distributed internal anomaly detection system for the Internet of Things is developed in [19]. The author's algorithm was not intended to run on low-capacity IoT nodes. The approach continuously monitors the packet size and data rate of one-hop neighbor nodes, looking for anomalies in network traffic. The model learns and infers normal behaviors based on the monitored data.

A resource-constrained IoT device-aware deep packet inspection method for detecting anomalies is investigated in [20]. The payload data is processed as a sequence of bytes, with the features selected as n-grams using a bit-pattern matching algorithm. The method is based on the similarity of payload data in IoT protocols. The authors ran tests on two Internet-connected devices and found that the false positive rates for worm propagation, tunneling, SQL code injection, and directory traversal attacks were meager, which is good news for people who use them.

In [21], the authors detected anomalous behavior in low-capacity 6LoWPAN networks by leveraging regular energy consumption as a

parameter. They defined lightweight power consumption models using the mesh-under and route-over routing schemes. The system sends out an alert and takes it off the routing table. False alarm rates, on the other hand, are reported.

The authors in [22] developed three algorithms for detecting wormhole attacks in the Internet of Things (IoT) networks. More specifically, their system algorithms detect a high volume of control packets being exchanged between (i) the tunnel's two ends; or (ii) a large number of neighbors forming to indicate an anomaly, resulting in a valid positive rate of 94% for detecting wormhole attacks and 87% for detecting the attacker node. Even though the system uses little power and memory, making it suitable for low-resource Internet of Things devices, the authors did not say the false-positive rates.

A PCA model to reduce the number of features and classifiers such as Softmax Regression and KNN was developed in [23]. The authors created an anomaly-based intrusion detection system (IDS) capable of real-time analysis in the Internet of Things (IoT) environments. The authors show that even though Softmax Regression resulted in a more straightforward and efficient system in terms of time and computing, the accuracy of the KNN model is 1% higher than Softmax Regression, according to experimental results using the KDD CUP 99 dataset.

In [24], a two-tier classification and dimension reduction mechanism for detecting malicious activity against an IoT backbone is proposed. The KDD CUP 99 dataset's 41 features were reduced to four dimensions. Following that, the samples were classified using Naive Bayes and kNN models. Their anomaly-based intrusion detection technique applies to probe, DoS, U2R, and R2L attacks in the IoT. Also, the authors reduced the number of features using PCA and Linear Discriminant Analysis (LDA). The method was evaluated on the NSL-KDD dataset and achieved an overall detection rate of 84.86% and a false alarm rate of 4.86%, respectively.

The paper at [25] presents Fed-ANIDS, a system combining Federated Learning (FL) and anomaly detection using autoencoders for enhancing network intrusion detection. It addresses privacy concerns in centralized models by using various autoencoder models, including simple, variational, and adversarial types, to compute intrusion scores based on reconstruction errors of normal traffic. Fed-ANIDS efficiently detects network intrusions using autoencoders while maintaining data privacy across distributed networks. Evaluated with popular datasets like USTC-TFC2016, CIC-IDS2017, and CSE-CIC-IDS2018, Fed-ANIDS shows high detection accuracy with fewer false alarms. The study highlights autoencoder-based models' superiority over GAN-based models in this context, underlining their effectiveness in threat detection while preserving data privacy in distributed networks.

The authors in [26] present an approach for anomaly detection in IoT networks using federated learning and deep neural networks (DNN). The study introduces a method that retains data privacy by keeping information localized on IoT devices while only sharing updated model weights with a centralized federated learning server. The paper demonstrates the efficiency of the DNN-based network intrusion detection system (NIDS) and compares its performance to traditional deep learning models. The approach shows improved model accuracy and reduced false alarm rates, highlighting the benefits of combining federated learning with deep learning in IoT environments. More specifically, a DNN-based anomaly detection method combined with federated learning is introduced to enhance privacy and efficiency in IoT networks. It utilizes the IoT-Botnet 2020 dataset for evaluation and demonstrates an improved model accuracy and reduced false alarm rate compared to conventional methods.

The paper in [27] explores the challenges of implementing Federated Learning (FL) in the Internet of Things (IoT) for anomaly detection. It highlights the limitations of FL due to data access constraints on devices, class balance issues, and device heterogeneity. The study investigates the application of data augmentation strategies to improve anomaly detection performance in IoT using three publicly accessible datasets. Key findings include up to 22.9% performance improvement

using data augmentation, particularly with stratified random sampling and uniform random sampling, over the baseline without data augmentation. The study also examines various data augmentation methods' effectiveness and computational cost, including Generative Adversarial Networks, in a federated learning context. The study addresses performance issues in FL due to class imbalance and device heterogeneity. The research demonstrates significant improvements in detection performance by employing various data augmentation methods, including random oversampling, stratified sampling, SMOTE, ADASYN, and Generative Adversarial Networks (GANs). The experiments used three publicly available IoT datasets, enhancing performance with a modest increase in computation time, particularly for random and stratified sampling methods.

The primary shortcoming of the abovementioned approaches is that they were rarely evaluated using IoT-specific datasets and/or communication protocols, except in [28]. The author used three methods, Extreme Gradient Boosting (XGBoost), GRU Recurrent Neural Networks, and LSTM Recurrent Neural Networks, to detect three types of attacks, namely denial of service (DoS), man-in-the-middle, and an MQTT-specific intrusion. Another major shortcoming is that they should have considered the power requirements of the algorithms, which is significant in the case of power-constrained IoT devices. To the best of our knowledge, this work is the first to propose an IDS for IoT that jointly considers the identification of unknown attacks (as anomalies) and classification in terms of performance and resource consumption while providing data privacy and security in intrusion detection along with the use of a custom FL approach for updating the whole MIIoT network in the hospital (ERP system). Our technique conducted a thorough evaluation of our technique's resource efficiency resulting in low power consumption, which is a pivotal aspect in IoT environments. This evaluation demonstrates that our approach is not only effective in terms of intrusion detection but also optimized for minimal energy usage, making it highly suitable for power-sensitive IoT applications. Additionally, regarding the dataset utilized in our study, we acknowledge its current unavailability to the public. However, to foster collaborative research and transparency in the field, we are in the process of making the dataset available upon request. This step ensures that interested researchers can access the data while we navigate the necessary protocols for wider public release.

Tables 1 and 2 present with the same columns names and meaning a comparative analysis of intrusion detection methodologies for MIIoT and IoT, respectively, as explored in the research study, with the recommended approach. In both tables, we are comparing Intrusion Detection Systems (IDS); the columns represent distinct elements essential to understanding and evaluating each IDS approach. The 'Reference' column lists the academic studies or papers, providing a context for each intrusion detection method. 'Detection Technique(s)' describes the specific algorithms or methods used to detect cyber threats, varying from machine learning approaches to other sophisticated techniques. The 'Detection Location' specifies where the intrusion detection system is implemented, such as in a specific device, network, or cloud-based setting. The 'Dataset' column is crucial as it mentions the data used for training and evaluating the IDS models, indicating the source and type of data. 'Attacks Identified' provides details on the range of cyberattacks that each system can detect, from specific threats like DDoS to a broader spectrum. The presence of 'Federated Learning (FL)' in an IDS approach is marked in its respective column, signifying its use in the study. 'Power Consumption Analysis?' is particularly relevant in MIIoT/IoT environments, indicating if the study assesses the energy efficiency of the IDS. The column '(U)known or (K)nown or (B)oth Attacks' classifies the studies based on their focus on detecting unknown, classifies known, or examining both types of cyberattacks. The 'ERP' column refers to 'Enterprise Resource Planning', showing if each study implements a complete solution. Finally, 'Custom FL' indicates if a tailored Federated Learning approach is implemented, catering to the specific needs of the IDS. Note that both tables are

structured to allow for a straightforward comparison of different IDS methods, highlighting their capabilities, focus on various attack types, and adaptability regarding power consumption and Federated Learning. The first table emphasizes MIIoT, while the second provides insights into broader IoT security, enabling a detailed understanding of the IDS landscape in these technologically interconnected areas. It is important to observe that due to the identical layout of both tables, our methodology primarily emphasizes MIIoT IoT and is hence presented in the first table. Although it is located in the first table, we can easily compare it with the other ways in the second table.

Tables 1 and 2 comprehensively compare various Intrusion Detection approaches in the context of MIIoT (Medical Internet of Things) and IoT security. Each row represents a different approach, including the reference paper, the detection technique employed, the detection location, the dataset used, and the types of attacks identified. Most of the reviewed approaches employ machine learning or deep learning techniques for intrusion detection. However, it is noteworthy that "GEMLIDS-MIIoT", presents a novel approach. This approach utilizes Support Vector Machines (SVM), Naive Bayes (NB), K-Nearest Neighbors (KNN), Decision Trees (DT), and Extreme Randomized Trees (ERT) for detection, making it a robust ensemble method. Moreover, it not only identifies various attacks, such as DDoS (Distributed Denial of Service), MitM (Man-in-the-Middle), Brute Force, and Scanning, but also efficiently handles power and identifies "Unknown Attacks". This holistic approach is a promising solution for enhancing MIIoT/IoT security.

Table 3 shows that GEMLIDS-MIIoT achieved an accuracy of 99.8% in the MIIoT IDS category. Comparing it to other MIIoT IDS approaches, it outperforms all of them, except DNN (99.9%) by 0.1%, including Hybrid CNN-LSTM (98.83%), XGBoost 97.83%, Random Forest Agent 97.21%, and HEKA (98.4%), making it one of the most accurate systems in this category. However, it is important to note that the IoT IDS category contains some approaches with higher reported accuracies, such as DNN (99.9%), AS-EBS (100%), and Intrusion Detection (100%). Therefore, in terms of accuracy, GEMLIDS-MIIoT is competitive in the MIIoT IDS category but not the highest among all IoT IDS approaches. However, our approach provides power consumption, reduced execution time, memory consumption, and unknown attack identification through anomaly detection (as shown in Section 5.2).

2.2. Background information

In this section, we provide the background information related to the following protocols and techniques examined in this research: IoT, MQTT, REST, Examined ML approaches in our investigation, Federated Learning, and Intrusion Detection.

2.2.1. IoT

The Internet of Things is a complex network with several interconnected components such as intelligent devices, gateways, communication protocols, Internet infrastructure, applications, cloud computing, and end-users. Smart devices generate and send data using dedicated communication protocols via an Internet-connected gateway. Subsequently, interactive apps process this data and deliver it to users through cloud-based platforms. These gadgets span a wide spectrum, ranging from basic temperature sensors utilized in smart homes to advanced drones designed for deployment in the defense industry. While IoT devices may have distinct agendas, they exhibit certain shared characteristics. The principal objective of an Internet of Things (IoT) device is to perceive and gather data about the physical world. Most Internet of Things (IoT) devices have limited memory capacity. Hence, these devices must minimize energy consumption, leading to the adoption of IoT communication protocols that function at restricted data rates across short distances. The Internet of Things (IoT) gateway facilitates the transmission of data gathered from IoT devices to applications using Ethernet or WiFi, employing TCP/IP protocols. The application

Table 1
Comparison of works in MIoT intrusion detection.

Reference	Detection technique(s)	Detection location	Dataset	Attacks identified	FL	Power consumption analysis?	(U)known or (K)nown or (B)oth attacks	ERP	Custom FL
[1]	Different ML (DT, SVM, K-MEANS)	The intrusion detection system on the connected medical device	The dataset is generated using a simulator. Not available	Not specified any attacks	X	X	K	X	X
[2]	Utilize stacked autoencoders for feature extraction and XG-Boost for classification and regression.	Propose an intrusion detection system (IDS) based on a stacked autoencoder for anomaly detection in a connected healthcare system (CHS)	The dataset is not publicly available	Interception, Forgery, and Tampering	X	X	K	X	X
[3]	N-frame for features extraction using KNN, SVM, RF and DT	Detection cyber-attack against PMD	The data are generated from different real devices. The dataset is not publicly available	Eavesdropping, DoS, MitM, Replay attack, and False data injection	X	X	K	X	X
[4]	Mobile agent-based intrusion detection system using ML and regression algorithms	Securing the network of connected medical devices	The dataset is not publicly available	DoS, Data falsification, and Passive listening	X	X	K	X	X
[5]	They employ PCA and GWO to reduce and dimensionalize data and DNN to classify it.	Intrusion detection system in MIoT system	Kaggle intrusion dataset	DoS, User-to-Root attack, probe attack, and Remote to local attack	X	X	K	X	X
[6]	Use of NB, DT, RF and XGBoost	For cyber-attack detection in MIoT networks, and IDS based on ensemble learning and a fog-cloud architecture is used.	Ton-IoT dataset	Not specified any attacks	X	X	K	X	X
[7]	CNN	IDS based on machine learning and multi-class classification designed for healthcare IoT in the smart city	The dataset is not publicly available	Firewall risk label: Normal, critical, major and minor	X	X	K	X	X
[8]	Authors compare the results of SVM, RF, KNN and ANN	IDS for healthcare using medical and network data	Dataset publicly available	MitM (spoofing and data alteration)	X	X	K	X	X
[9]	EOS-ELM	Framework for attack detection in the Fog node	NSL-KDD dataset	MitM and DDoS	X	X	K	X	X
[10]	CNN to extract features and LSTM to classify data	Hybrid deep learning-based model for malware detection in the MIoT deployed at the SDN plane application level	not mentioned	Not specified any attacks	X	X	K	X	X
[11]	Transformer-based AE and VAE with SVDD	Digital healthcare IoT	ECG Data	Anomalies Detection	✓	✓	U	X	X
GEMLIDS-MIOT	Examines the classification using Support Vector Machine, Naive Bayes, K-Nearest Neighbors, Decision Tree and Random Forests. Finally, it uses for the identification the one-class SVM and for the classification the Random Forests along with FL for updating the Enhanced Random Forest at sinkholes	At the sinkhole	Data generated using emulation and CICEV2023	Distributed Denial of Service (DDoS) attack, Man-in-the-Middle (MitM) attack, Brute Force Authentication attack, Network scanning attack	✓	✓	B	✓	✓

Table 2

Comparison of IoT intrusion detection works along with the paper from literature review.

Reference	Detection technique(s)	Detection location	Dataset	Attacks identified	FL	Power consumption analysis?	(U)known or (K)nown or (B)oth attacks	ERP	Custom FL
[12]	Suricata IDS	Central host	Not specified	DoS	X	X	K	X	X
[13]	Deep Auto Encoder, DFFNN	IIoT/IICs	NSL-KDD, UNSW-NB15	Various	X	X	K	X	X
[14]	Three-pattern detection (AS-EBS)	Raspberry Pi	Not specified	Not specified	X	X	K	X	X
[15]	Random Neural Networks, LSTM	IoT networks	Virtual network data	SYN flooding	X	X	K	X	X
[16]	ANN, J48 DT and Naive Bayes (Hybrid)	IoT networks	Not specified	Various	X	X	K	X	X
[17]	Semi-supervised learning ESFCM with ELM	Fog-based IDS	NSL-KDD dataset	Various	X	X	K	X	X
[18]	Anomaly detection (Bot Analysis)	6LoWPAN networks	Not specified	Wormhole	X	X	K	X	X
[19]	Anomaly detection (MGSS, RSS and ISS)	IoT	Not specified	Various	X	X	K	X	X
[20]	Ultra-lightweight deep-packet anomaly (Deep packet inspection)	IoT	Internet-connected devices	Various	X	X	K	X	X
[21]	Intrusion detection	6LoWPAN networks	Not specified	Various	X	X	K	X	X
[22]	Heuristic algorithm	IoT networks	Not specified	Wormhole	X	X	K	X	X
[23]	KNN (Feature reduction)	IoT	Not specified	Various	X	X	K	X	X
[24]	TDTC (Dimension reduction in IDS)	IoT backbone	NSL-KDD dataset	Probe, DoS, U2R, R2L	X	X	K	X	X
[25]	Federated anomaly-based NIDS	IoT networks	USTC-TFC2016, CIC-IDS2017, CSE-CIC-IDS2018	Various	✓	X	U	X	X
[26]	Federated DNN-based NIDS	IoT networks	–	Network intrusions	✓	X	U	X	✓
[27]	Data augmentation in federated learning	IoT devices	USTC-TFC2016, CIC-IDS2017, CSE-CIC-IDS2018	Various, with focus on anomaly detection	✓	B	X	X	X

Table 3

Accuracy of investigated IDS approaches.

Category	Authors	Proposed approach/Accuracy
MIoT IDS	Gao et al. [1]	DT pruned 90.37%
	He et al. [2]	XGBoost 97.83%
	Newaz et al. [3]	HEKA 98.4%
	Odesile et al. [4]	Random Forest Agent 97.21%
	Swarna et al. [5]	DNN 99.9%
	Kumar et al. [6]	E-ADS 96.352%
	Lee et al. [7]	M-IDM 96.50%
	Hady et al. [8]	RF (EHMS) 92.27%
	Alrashi et al. [9]	OS-ELM (FBAD) 98.19%
	Khan et al. [10]	Hybrid CNN-LSTM 98.83%
	Raza et al. [11]	AnoFed with Transformer-based AE and VAE, SVDD, ECG Anomaly accuracy 98.125%
IoT IDS	GEMLIDS-MIOT	one-class SVM 99.7% ERF 99.8%
	Kasinathan et al. [12]	DoS decoder in Suricata Unknown%
	Muna et al. [13]	Deep Feed Forward Neural Network (DFFNN) 98.6%
	Oh et al. [14]	AS-EBS 100%
	Evmorfos et al. [15]	Random Neural Network (Gelembe) 80.7%
	Soe et al. [16]	ANN, J48 DT and Naïve Bayes (Hybrid) 99.10%
	Rathore et al. [17]	ESFCM with ELM 86.53%
	Cho et al. [18]	Bot Analysis Unknown%
	Thanigaivelan et al. [19]	MGSS,RSS and ISS Unknown%
	Summerville et al. [20]	ultra-lightweight deep-packet anomaly Unknown%
	Lee et al. [21]	Intrusion detection 100%
	Pongle et al. [22]	Heuristic algorithm 94%
	Zhao et al. [23]	KNN 84.406%
	Pajouh et al. [24]	TDTC 84.86%
	Wang et al. [29]	FDL MI-DNN, Avg. Accuracy 98.51%, F1-score 97.53%, TPR 98.44% and TNR 97.31%
	Idrissi et al. [25]	Fed-ANIDS & AE,Avg. F1-score: 82.42%, FDR: 0.55% and Accuracy: 92.60%
	Weinger et al. [27]	DA in FL, improve by 22.9%

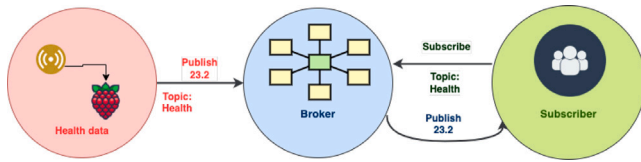


Fig. 1. MQTT Publish/Subscribe model.

procedure involved collecting data about the physical environment, aiming to extract valuable information that can be utilized for decision-making or the remote operation of physical objects. [30]. MQTT and REST are two of the most commonly used application layer protocols on the Internet of Things.

2.2.2. MQTT

Message Queuing Telemetry Transport (MQTT) facilitates the dissemination of telemetry information from network clients with limited resources to IoT devices characterized by significant latency. The communication protocol adheres to a publish–subscribe pattern and is employed for machine-to-machine (M2M) communication. The protocol under consideration is designed to cater to the needs of small sensors and mobile devices. Its lightweight nature characterizes it and is optimized explicitly for networks with high latency or unreliability. This protocol operates on top of the TCP/IP stack. MQTT clients can function as publishers or subscribers, contingent upon their role in transmitting or subscribing to receive them. It is feasible to incorporate both of these functions into a single MQTT client. When a client requires transmitting data to a broker, it is referred to as a publisher, and the corresponding procedure is denoted as a publish operation. The act of receiving data from the broker by the client is referred to as a subscription. EMQX is a cloud-native, distributed MQTT broker that operates on an open-source platform. Its primary function is facilitating communication between clients, who initiate message transmission, and subscribers, who receive these messages. This paper is a broker within our Internet of Things (IoT) system. MQTT X is a client subscribing to a MQTT subject on the broker's side, enabling the broker to automatically receive all incoming information associated with the specified MQTT topic. Fig. 1 shows an example of the publish–subscribe architecture of MQTT that sends health-related data across the topic “Health”. Note that the publisher devices transmit data to a broker, and the subscribers receive data for the topics they have subscribed to from the broker [31].

2.2.3. REST

REST (representational state transfer) was developed to guide the design and development of the architecture of the World Wide Web. The Representational State Transfer (REST) framework offers a set of principles and standards for the design of distributed hypermedia systems, specifically those operating on an Internet scale, such as the World Wide Web. The REST architectural style emphasizes several key principles, including the scalability of interactions between components, the use of uniform interfaces, the possibility for components to be deployed independently, and the construction of a layered architecture. These principles enable the implementation of caching components, reducing the latency perceived by users. Additionally, REST supports the enforcement of security measures and the encapsulation of legacy systems. A REST API, sometimes called a RESTful API, is an application programming interface (API) or web API that conforms to the limitations of the REST architectural style. It facilitates the interaction with RESTful web services. The architectural style in question can employ SOAP and protocols like HyperText Transfer Protocol (HTTP). This technology necessitates a reduced amount of bandwidth and exhibits compatibility with distinct data types such as plaintext, HTML, XML, JSON, and others [32]. Moreover, HTTP clients often use the

Transmission Control Protocol (TCP) to establish a connection through a three-way handshake mechanism, ensuring secure server communication. The data generated by the Internet of Things (IoT) system is transmitted over a Representational State Transfer (REST) web service to a server in order to protect its secrecy. The HTTP service is the intermediary software layer employed by the Internet of Things (IoT) system for transmitting data to the server.

2.2.4. MIoT attacks

Overall, the vulnerability of the MIoT system to different attacks arises from its incorporation of wireless connection and external equipment for the purpose of sensor control and enhancement. The open literature has put forth two separate architectural approaches for the MIoT: single-hop and multi-hop. The single-hop architecture is predicated on the utilization of sensors for gathering and transmitting data. Nevertheless, this architectural design is susceptible to a singular point of failure, wherein the failure of a single component inside the personal server layer can jeopardize the integrity and functionality of the entire MIoT system. Utilizing a multi-hop architecture enables the collection and transmission of data by sensors while simultaneously facilitating data routing. This architectural approach offers advantages such as node mobility and reduced energy consumption during the process of data transfer. Consequently, the architectural framework in question may be susceptible to routing vulnerabilities inherent in wireless sensor networks. The following are the most common types of network and system attacks:

- Attack at data collection level: Due to the MIoT system's integration of wireless communication and external equipment for controlling and upgrading sensors, the MIoT is a single-hop and multi-hop architecture vulnerable to various attacks.
- Attack at the transmission level: At the transmission level, there is a significant threat risk, as wireless communication enables an attacker to intercept, modify, or block messages sent and exchange valuable information about the patient's condition.
- Attack at storage level: All information about a patient's health condition, treatment, and identity is stored at this level, making it an attractive target for adversaries seeking access to these data.

The following are the attacks that our investigation identifies:

- **Man-in-the-Middle (MitM) Attack [33]:** The Man-in-the-Middle (MitM) attack refers to a form of cyberattack when an attacker secretly intercepts and potentially modifies the communication between two entities, unbeknownst to them. The potential consequences include the unauthorized acquisition of data, interception of communications, or the unauthorized takeover of a session. Man-in-the-middle (MitM) attacks capitalize on weaknesses present in communication protocols or network setups.
- **Distributed Denial-of-Service (DDoS) Attack [34]:** DDoS attacks encompass the deliberate act of flooding a specific system or network with an excessive volume of traffic originating from numerous sources, thereby incapacitating its functionality and leaving it unreachable. Malicious actors frequently employ botnets to execute distributed denial of service (DDoS) attacks, interrupting services and causing financial detriment for targeted entities.
- **Brute Force Authentication Attack [35]:** A brute force attack is an iterative approach employed to ascertain a password or encryption key by exhaustively methodically examining all conceivable combinations. The method above is a frequently employed means of illicitly obtaining entry into user accounts, systems, or networks.
- **NMAP Identification Attack [36]:** NMAP is a network scanning program that malicious actors employ to discern accessible ports, services, and potential weaknesses on designated computers—frequently employed as a reconnaissance technique before the

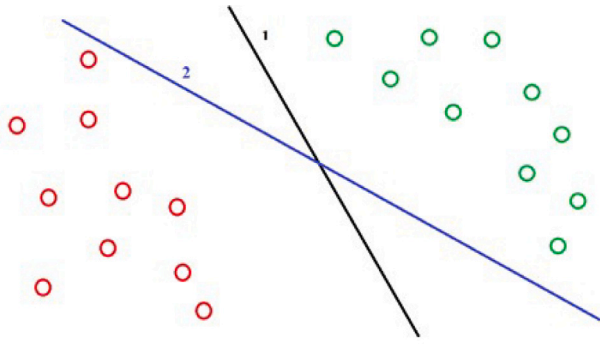


Fig. 2. Classification by SVM.

initiation of more advanced attacks. The identification of NMAP scans holds significant importance in the realm of network security.

2.2.5. Examined ML models

This section briefly overviews the algorithms used in our traffic identification and attack classification approach.

Support Vector Machines (SVM) [37]: The Support Vector Machines (SVM) technique is widely utilized in the field of supervised machine learning to solve classification problems. The program uses a kernel trick to extract pivotal information from the dataset. The method aims to ascertain an optimal boundary and divide the data based on the given labels. This essential distinction will help the algorithm figure out the new data's best position. The provided illustration in Fig. 2 depicts an instance of categorization using Support Vector Machines (SVM). The provided illustration in Fig. 2 is an example for consideration. The black line represents one classifier, while the blue line represents another. Upon evaluating the margins, which refer to the extent of separation from the training data, it becomes evident that the model represented by “one” has superior data classification capabilities. The SVM classifier will exert significant effort to optimize the margins for distinguishing between different classes. It should be noted that in our inquiry, the One-Class SVM was employed as the classifier for detecting anomalies and unknown assaults/attacks (we use the SVM in our research to identify unknown attacks), as suggested by other studies in the field, for a small and large data set [26,38–41]. More specifically, the One-Class Support Vector Machine (SVM) is a variant of the regular Support Vector Machine that is designed primarily to identify abnormalities or innovations. The specialization of this technology enables it to identify patterns in data that deviate from recognized norms, resulting in exceptional effectiveness in detecting unexpected cyber threats. The One-Class SVM differs from typical SVMs in that it is exclusively trained on data that exhibits ‘normal’ behavior rather than being used for classification tasks involving several classes. This quality is crucial for its effectiveness in identifying unexpected attacks, as it does not require prior knowledge of all possible attack types. During training, the One-Class SVM algorithm aims to create a boundary in the feature space that encompasses most of the ‘normal’ data points. The border is constructed to include the region where the probability of encountering a typical data point is substantial while still providing some flexibility by eliminating a small number of ‘normal’ data points outside the boundary. This guarantees that outlier values within the regular dataset do not unduly affect the boundary. When dealing with data that cannot be separated in a linear manner, the One-Class SVM employs the kernel method to convert the input data into a space with more dimensions. This facilitates the definition of a clear border. Following the completion of training, the model possesses the capacity to identify anomalies or distinctive patterns. It identifies new data points that differ from the decision boundary as likely anomalies or unrecognized attacks.

The One-Class SVM excels at identifying unforeseen attacks by using its proficiency in anomaly detection, the adaptability of the kernel approach, its resilience against overfitting, and its capability to handle high-dimensional data. However, the quality and comprehensiveness of the ‘normal’ data used during training significantly impact the model's efficacy [26,38–41].

In selecting the One-Class Support Vector Machine (SVM) for our study, our decision was significantly informed by a thorough literature review and the insights gleaned from the papers [26,38–41]. Collectively, these papers underscore the effectiveness of One-Class SVM in anomaly detection, especially in contexts where anomalies are sparse and not well-represented in the data. This method stands out in its ability to model the ‘normal’ behavior of data, thereby efficiently identifying deviations as anomalies. Unlike KNN or Centroid Classifiers, which may require substantial and balanced datasets to achieve high accuracy, One-Class SVM is particularly adept at working with datasets where anomalies are rare. This characteristic aligns closely with our research objectives, where detecting infrequent and potentially novel anomalies is crucial. The literature review highlighted the strengths of One-Class SVM in handling such data scenarios, making it a more suitable choice for our study's focus on anomaly detection.

K-Nearest Neighbors (KNN) [42]: The K-nearest neighbors (KNN) algorithm operates under the assumption that data points belonging to the same class tend to be located in close proximity to each other. The fundamental concepts that characterize the K Neighbors classifier are proximity, distance, and closeness. The metric parameter is utilized to elucidate the methodology employed in calculating this proximity. The technique continues by using the Minkowski distance (as the distance, the approach can also use the Euclidean distance). The characterization of the phenomenon can be achieved by employing the formula shown below (Formula (1)):

$$\text{Minkowski Distance} = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (1)$$

It assigns a class to an object by considering the class of its nearest objects' classes (K is a small positive integer).

Decision Tree (DT) [43]: The decision tree classifier is a classification system that is built on a tree structure comprising two distinct types of nodes: decision nodes and leaf nodes. Decision nodes serve as checkpoints that correspond to specific feature values and have the ability to branch into many branches. The leaf nodes in a decision tree represent the final outcomes resulting from the preceding decisions, and they do not have any more branches. The process involves evaluating several options, thoroughly analyzing each option, and eliminating extraneous pathways that the classifier may pursue. At each process level, an attribute test condition is selected and incrementally constructed starting from the top node. Leaf nodes, which are capable of receiving values from a discrete set, can be employed for the purpose of classification. Decision tree classifiers utilize many metrics such as EP, Gini Impurity, Information Gain, Variance Reduction, and Measure of Goodness to assess the accuracy of the tree.

Naïve Bayes (NB) [44]: Naïve Bayes is a probabilistic classifier derived from applications of Bayes' Theorem as stated below:

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)} \quad (2)$$

where A , B are events.

$P(A/B)$ is probability of A given B is true

$P(B/A)$ is probability of B given A is true

$P(A)$, $P(B)$ are independent probabilities.

An advantage of using the Naïve Bayes classifier is that the process of maximum likelihood training can be done in linear time complexity compared to its counterparts, which may not offer the same linear time possibility. Naive Bayes is a conditional probability model that

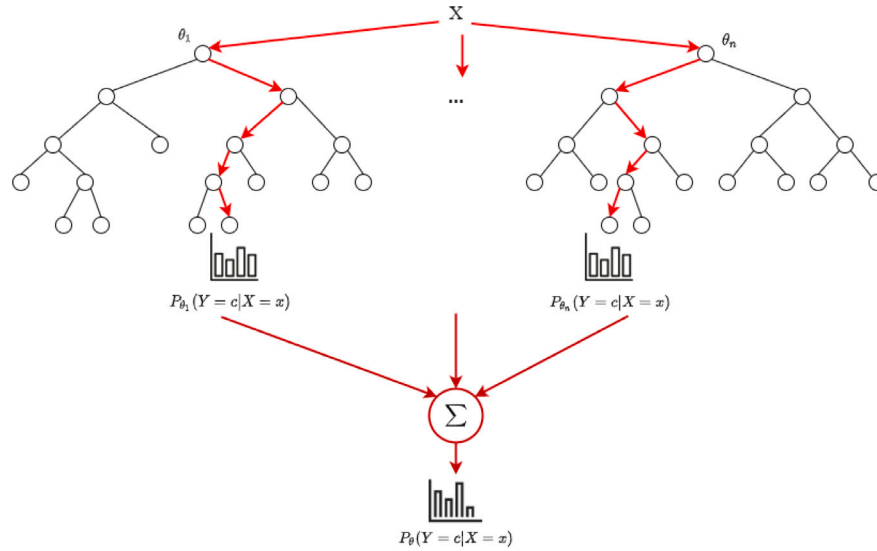


Fig. 3. Classification by Random Forest.

tries to assign a class C_k by calculating the probability of the class C_k given a feature x from a set of n such features. Naive Bayes Classifiers are classified into the following categories: (i) Multinomial Naive Bayes: The classifier uses the frequency of words in the document as features/predictors; (ii) Bernoulli Naive Bayes: Similar to multinomial naive Bayes, but the predictors are boolean variables; (iii) Gaussian Naive Bayes: The predictors take on a continuous value and are not discrete; we assume that these values are sampled from a Gaussian distribution. In this work, we use the Gaussian Naive Bayes classifier.

Random Forest (RF) [45]: The Random Forests classifier is a descendant of the Decision Tree classifier. It consists of a large number of distinct decision trees. When multiple decision trees provide distinct class values for the same object, the class that receives the most votes is chosen as the final class assigned to the object. The critical point is that these individual decision trees are highly correlated with one another. As a result, the Random Forest is extremely powerful. Because the correlation between the two trees is new, an error induced in one tree may not affect the other. Randomness is required in these uncorrelated models to produce the correct class value for the object. Bagging and feature randomness are techniques for ensuring a high level of randomness. More precisely, In RF, each decision tree is built using a random subset of the training data, often selected with replacement (a method known as bootstrapping). Additionally, when splitting nodes during the tree-building process, a random subset of features is considered at each split. This method introduces diversity among the trees, as each tree sees different parts of the training data and different sets of features. However, despite this diversity, there is an underlying correlation among the trees due to the shared pool of training data and features from which they are drawn. This correlation is beneficial in terms of the power of RF. When the individual decision trees in the RF make their predictions, these are then aggregated (usually through a majority vote for classification tasks or averaging for regression tasks). Aggregating predictions from multiple, somewhat correlated trees makes the RF more powerful and accurate than individual decision trees. This is because the errors of individual trees are likely to be different and, when averaged, can cancel each other out, leading to a more accurate final prediction. In summary, the correlation among decision trees in RF refers to their shared origins regarding data and features despite their individual variations. This correlation, combined with the aggregation of their predictions, contributes to the overall strength and robustness

of the RF algorithm, particularly in handling diverse and complex datasets [46].

A Random Forest uses the outputs of all trees in the forest to determine the majority decision when classifying a particular data instance. By combining the outputs of multiple trees, the classifier becomes more robust than decision trees, which frequently suffer from the over-fitting problem. More specifically, the Random Forest model is constructed by combining numerous decision trees, each trained on various subsets of the data and features. This approach enhances the model's resistance to overfitting and results in a high level of accuracy. A Random Forest (RF) is a machine learning model that constructs an ensemble of decision trees, named a forest, such that each decision tree is built using an independently and identically distributed random vector [47]. For classifying a particular data instance, a Random Forest uses the outputs of all trees in the forest to pick the majority decision. The utilization of the results of multiple trees makes the classifier more robust than decision trees, which suffer from the over-fitting problem in many cases.

At a high level, the RF algorithm works as follows:

1. The complete training set S consisting of n data instances with class labels $\{c_i, i = 1, \dots, n\}$ from a set of classes C is split into K random subsets using bootstrap sampling:

$$S = \{S_1, S_2, \dots, S_K\}$$

2. A random feature vector θ_k is created and used to build a decision tree from each S_k . All $\{\theta_k, k = 1, 2, 3, \dots, K\}$ are independent and identically distributed.
3. Each tree $r(S_k, \theta_k)$ is grown without pruning to form the forest R .
4. The classification of a test data instance x is calculated as follows:

$$H(x) = \operatorname{argmax}_{C_j} \sum_{i=1}^K I(h_i(x) = C_j) \quad (3)$$

where I is the indicator function, and $h_i(x)$ is the result of classification by $r(S_i, \theta_i)$.

Fig. 3 shows a simplified view of classification by Random Forests. Information gain is a commonly used metric for deciding the splitting criteria for the various nodes in the decision trees. The information

gained from the split of a node S based on a random variable a is calculated as follows:

$$IG(S, a) = E(S) - E(S|a) \quad (4)$$

Here, $E(S)$ is the entropy of the parent node before the split, and $E(S|a)$ is the weighted average of the entropies of the child nodes after the split. $E(S)$ is calculated as:

$$E(S) = - \sum_{i=1}^C p(c_i) \log p(c_i) \quad (5)$$

where $p(c_i)$ is the probability of a data instance in node S having class label c_i .

The computational complexity of the algorithm in Big-O notation [48], is commonly denoted as $O(K \times N \times \log(N) \times F)$, where K represents the number of trees in the forest, N denotes the number of training examples, $\log(N)$ approximates the depth of the trees, and F signifies the number of features. The Random Forest algorithm constructs numerous decision trees during the training process. Every tree is constructed using a bootstrapped sample of the data, and during each split in the tree, a random selection of features is taken into account. This approach incorporates stochasticity into the model, effectively mitigating variance and preventing overfitting, a prevalent concern associated with decision trees. The ultimate forecast of the Random Forest is achieved by combining the forecasts of individual decision trees, usually through majority voting for classification problems or averaging for regression jobs [49]. The computational time is affected by the number of classes in the dataset, particularly during the tree construction and voting phases. However, the overall computational complexity order is determined by the number of trees, the size of the training dataset, and the number of features, regardless of the number of classes [49].

Enhanced Random Forest (ERF) [50]: Researchers have proposed several enhancements to the Random Forest algorithm to improve its performance and versatility. The Enhanced Random Forests enhancements we include in our proposed solution are the following:

- **Feature selection based on importance scores [51]:** Feature selection is a critical preprocessing step in building effective Random Forest models. Each feature's importance is quantified in Random Forest based on its contribution to the model's predictive performance. This process assigns an importance score to each feature, reflecting its ability to discriminate between classes or predict the target variable. Features with higher importance scores are considered more influential in making predictions and are often retained, while less important features may be excluded from the model. Feature selection based on importance scores reduces dimensionality and enhances model interpretability and training efficiency. Random Forest models can improve accuracy and generalization on various classification and regression tasks by focusing on the most informative features. This technique helps identify and prioritize relevant input variables, ensuring the model's predictive power is harnessed effectively.
- **Handling imbalanced datasets [52]:** is a pivotal aspect of improving the performance of Random Forest models, especially when dealing with real-world datasets where one class significantly outnumbers the others. Imbalanced datasets can lead to biased models that favor the majority class while neglecting the minority class. Chen et al. [52] proposed techniques to address this issue within the Random Forest framework. One common approach is to assign different class weights to each class, penalizing misclassifications of the minority class more than the majority class. Additionally, techniques such as oversampling the minority class or undersampling the majority class can be employed to balance class distributions within each forest decision tree. By doing so, Random Forest models become more adept at recognizing patterns and making accurate predictions for all classes, making them a robust choice for imbalanced datasets.

- **Balancing class weights [53]:** In the context of Random Forest, handling imbalanced datasets is a critical challenge. When dealing with classification tasks where one class significantly outnumbers the others, the model can become biased towards the majority class, leading to poor performance in identifying minority class instances. To address this issue, the technique of balancing class weights is employed. By assigning higher weights to minority class samples during the tree construction and voting process, Random Forest can provide more equitable consideration to all classes. This adjustment ensures that the model is not overly influenced by the majority class, making it more capable of detecting rare or critical events. The approach effectively enhances the model's sensitivity to minority class instances, improving its overall predictive accuracy and reliability.
- **Hyperparameter tuning and optimization [54]:** Another crucial enhancement in the realm of Random Forest is the systematic tuning of hyperparameters. Fine-tuning hyperparameters such as the number of trees, maximum depth of trees, minimum samples per leaf, and learning rates can significantly impact model performance. Optimization techniques like grid search, random search, or Bayesian optimization are employed to find the best combination of hyperparameters. This process helps tailor the Random Forest model to the specific characteristics of the dataset, leading to improved accuracy and robustness.

The optimizations described above in the context of the proposed enhanced random forest are implemented in the subsequent portions of this study. (i) Section 5.1.1 is dedicated to the optimization of “Feature selection based on importance scores”; (ii) Section 5.1.2 encompasses the optimizations of “Handling imbalanced datasets” and “Balancing class weights”; and (iii) Section 5.2.3 focuses on the optimization of “Hyperparameter tuning and optimization”.

In our study, we employ the Enhanced Random Forest (ERF) as a distinct model, specifically adapted for the challenges in Medical Internet of Things (MIoT). ERF, an advanced version of the traditional Random Forest algorithm, is optimized for MIoT applications. It incorporates specific enhancements for handling high-dimensional data and imbalanced datasets, which is common in MIoT environments. These modifications include techniques for efficient data processing and improved learning capabilities, such as feature selection, dimensionality reduction, synthetic data generation, and weighted sampling [50,55]. Moreover, ERF is designed to be computationally efficient and power-conserving, addressing the resource limitations and power constraints of MIoT devices. This is achieved through optimizations like pruning strategies and efficient tree construction algorithms [56,57]. Studies in various fields, including industrial fault classification and medical imaging, have demonstrated ERF's effective application, underscoring its suitability for MIoT scenarios [58]. This paper also distinctly identifies ERF and its specialized adaptations, highlighting its significance and applicability in the MIoT context. These adaptations are not merely general improvements but tailored to address specific MIoT challenges, enhancing the model's effectiveness in real-world MIoT security solutions.

In the development of our Enhanced Random Forest (ERF) algorithm, we adhere to the standard Big O notation [48] for computational complexity, typically denoted as $O(K \cdot N \cdot \log(N) \cdot F)$ for Random Forests. In this expression, K is the number of trees, N the number of training instances, $\log(N)$ the approximate depth of the trees, and F the number of features. Thus, by considering the Big-O notation and the facts that K and F (which is 25 in our case) are constants, the resulting Big-O notation for the ERF should be $O(N \cdot \log(N))$. As an example, according to Table 7, our ERF model, after balancing the dataset through oversampling, trains on 19,208 instances across 25 features, leading to a complexity of $O(K \cdot 19208 \cdot \log(19208) \cdot 25)$, which simplifies to $O(19208 \cdot \log(19208))$. While the model handles five different classes, this factor primarily influences computational time rather than theoretical complexity, which governs the forest size, training set size, and feature count.

2.2.6. Federated learning

Federated Learning enables multiple users to collaboratively train a machine learning model without disclosing their individual local datasets. Federated Learning (FL), commonly called collaborative learning, is a machine learning methodology that facilitates the training of an algorithm on several decentralized edge devices or servers without the need to transmit local data samples. This strategy contrasts with traditional centralized machine learning techniques, in which all local datasets are uploaded to a single server, and with conventional decentralized tactics, which normally presume that local data samples are uniformly distributed. Federated Learning (FL) facilitates the collaboration of multiple entities in constructing a consistent and resilient machine-learning model without the need to exchange data. This approach effectively tackles significant concerns, including privacy, security, access rights, and the heterogeneous nature of data access [59,60]. The following types of FL exist:

- **Federated centralized Learning:** In the context of federated learning, a centralized approach involves utilizing a central server to effectively manage and synchronize all nodes involved in the learning process. During the initial stage of the training process, the server assumes the responsibility of selecting the nodes and consolidating the received model modifications. The potential for the server to become the system's bottleneck arises from requiring all selected nodes to update a single entity [61].
- **Federated decentralized learning:** In a decentralized, federated learning (FL) environment, the nodes can coordinate themselves to obtain the global model autonomously. This configuration mitigates the risk of single-point failures as model changes are exclusively transmitted to networked nodes, hence removing the central server. Nevertheless, the efficiency of the learning process may be hindered by the network structure [61].
- **The Federated Heterogeneous Learning:** Various application industries make use of heterogeneous clients, including mobile phones and Internet of Things (IoT) devices. Contemporary FL methodologies operate under the assumption that the local and global model architectures are identical [62].

Note that in our approach, we use Federated Heterogeneous Learning.

Federated learning parameters In this section, we show the different parameters that optimize learning in the control of the FL process; these are the following:

- Number of rounds of federated learning: R
- Total number of nodes utilized: N
- Fraction of nodes utilized throughout every iteration for each node: FN
- Local batch size utilized during each iteration of learning: BS
- Number of local training iterations before pooling: I
- Local education rate: η

These settings must be optimized based on the machine learning application's restrictions (e.g., available computing power, memory, bandwidth).

Federated learning aggregation method In federated learning, the aggregation method is crucial in combining the model updates from different devices while preserving privacy and ensuring model convergence. Several aggregation methods are used in federated learning, and the best type depends on the specific application and requirements. According to the literature, the following aggregation methods exist:

- **Federated Averaging (FedAvg)** [63]: Federated averaging stands as a highly prevalent aggregation technique within the realm of federated learning. In this methodology, each device updates its own local models by utilizing its own data and then

transmitting the updates to the central server. Subsequently, the server performs computations to derive the weighted average of the model updates, so generating a global model. From a mathematical perspective, the process of aggregation can be mathematically described as follows:

$$\theta_{\text{global}} = \frac{1}{N} \sum_{i=1}^N w_i \cdot \theta_{\text{local}}^i \quad (6)$$

where θ_{global} is the global model, N is the number of devices, w_i is the weights assigned to each device (usually based on the amount of data or device reliability), and θ_{local}^i are the local model updates.

- **Federated Learning with Secure Aggregation (FedSecAgg)** [64]: Secure aggregation methods are utilized in situations where maintaining privacy and security are of utmost importance. The FedSecAgg framework employs cryptographic methods, such as secure multi-party computation (MPC), to combine model updates while maintaining anonymity. Every individual device employs encryption to secure its model update, ensuring that the server cannot view the raw updates while performing the aggregate process. This measure guarantees the confidentiality of the individual updates.
- **Federated Quantization** [65]: In federated quantization, the process involves initially quantizing or compressing the model updates at the local devices to minimize the communication cost. The quantized changes are subsequently transmitted to the central server, which is consolidated and used to recreate the global model. This methodology effectively mitigates communication expenses while upholding a satisfactory level of model precision.
- **Personalization and Differential Privacy** [66]: Some federated learning applications require personalized models for each device while ensuring differential privacy. In such cases, aggregation methods need to balance personalization and privacy. Aggregation techniques that allow for customization of the global model while incorporating privacy-preserving mechanisms are employed.

In realizing our approach, we used the “Federated Learning with Secure Aggregation (FedSecAgg)” aggregation due to its security through encryption, private key cryptography, and the digital signature.

Iterate through the decision trees under enhanced random forests using depth first search for federated learning delta calculation The Depth First Search (DFS) traverses trees and graphs commonly used in data structures. Implementing recursion and data structures, such as dictionaries and sets, can be easily accomplished. The Depth-First Search (DFS) Algorithm is presented in the following manner: (i) Select a node from the given set of nodes. (ii) If the selected node has not been visited, mark it as visited. (iii) Recursively apply the same process to all neighboring nodes of the selected node. (iv) Repeat steps (ii) and (iii) until all nodes have been visited or the desired node has been found.

The time complexity and Big-O notation [48] for Depth-First Search (DFS) on a network or tree is commonly expressed as $O(V+E)$, where V represents the number of vertices and E represents the number of edges. Furthermore, it is worth noting that the temporal complexity of Depth-First Search (DFS) on a tree is $O(N)$, with N representing the total number of nodes within the tree. The average time complexity arises from the fact that the average time complexity of a set insertion operation is $O(1)$. In contrast, the work would become more intricate if a list were employed. The average time complexity arises from the fact that the average time complexity of a set operation is $O(1)$. If a list were used instead, the complexity of the task would be more intricate. Therefore, the whole computational complexity can be expressed as $O(n)$.

The outcomes of a depth-first search are shown in the paragraph. More specifically, the result of doing a Depth-First Search (DFS) on a

tree is a sequentially arranged collection of vertices, the specific order of which is contingent upon implementing the DFS algorithm. The concept of vertex order is elucidated in the subsequent paragraph. The subsequent section illustrates the many vertex orderings that can arise in a Depth-First Search (DFS) algorithm. In a more specific context, the depth-first search algorithm can be employed to linearly arrange the vertices of a graph or tree in a sorted manner. Four distinct methods exist through which this task can be achieved.

- **Preordering:** A preordering is a list of the vertices in the order in which the depth-first search method first visited them. This is a concise and natural approach to describe the search's progress, as was done previously in this article. A Polish notation expression is a preordering of an expression tree.
- **Post-order:** A post-order is a list of vertices in the order in which the algorithm last visited them. A post-ordering of an expression tree corresponds to the expression in Polish notation inverted.
- **Reverse Preordering:** A reverse preordering is the opposite of a preordering: a list of the vertices in the reverse order of their initial visit. Reverse preordering differs from post-ordering.
- **Reverse Post-order:** A reverse post-ordering is the opposite of a post-ordering: a list of the vertices in the reverse order of their previous visit. Reverse post-ordering differs from preordering.

The DFS algorithm will calculate the Enhanced Random Forest difference between the old and new model by subtracting the traversal result set of the old model from the new model for the FL to be used and inform the rest of the devices through the cloud model with the delta changes (more details can be found at Section 4.3).

2.2.7. Intrusion detection

Intrusion Detection detects rare or abnormal events in a network by monitoring its traffic and hosts. An intrusion detection system (IDS) attempts to detect malicious activity an attacker generates on an organization's information technology infrastructure. In general, intrusions attempt to gain unauthorized access to a device or system or to cause a denial of service attack on it [67]. An IDS could be network-based (NIDS) or host-based (HIDS). A network intrusion detection system (NIDS) monitors only network traffic and analyses packet headers, payloads, and statistics in order to detect malicious activity. On the other hand, a HIDS is installed on a device and monitors host traces (file system changes, system calls, running processes, and so forth) and network traffic to detect abnormal behavior. When the analysis engine detects an intrusion attempt, the IDS records pertinent investigation data and notifies security analysts. Three primary techniques for detecting intrusions are signature-based, anomaly-based, and specification-based. A brief description of the detection methods is shown below:

- Signature-based intrusion detection systems (IDS) are used to monitor network attacks by looking for certain patterns. This makes them vulnerable to zero-day attacks, and the IDS's ability to adapt to each device is not very good, so they are at risk.
- Anomaly-based methods use many machine learning techniques to get a general idea of how the system usually works and then use that knowledge to look for things that are not normal.
- A hybrid of these two approaches (named specification-based intrusion detection) is demonstrated in [68], in which manually specified behavioral program specifications are used to detect attacks. This approach has been proposed as a promising alternative that combines the strengths of misuse detection (accurate detection of known attacks) and anomaly detection (ability to detect novel attacks). One of the main benefits of this hybrid approach is that it balances the storage costs of signatures with the intensive computational tasks that come with learning-based methods.

An IDS for IoT networks can be deployed in two ways: connected to the gateway router or embedded in each IoT device. The benefit of embedding the IDS in the gateway router is that it enables centralized detection and management [12,68]. Internet-based attacks on IoT devices could be detected at a single point and the most basic level. The downside is that it slows down communication between IoT devices and the gateway because the IoT IDS has to check on the network states of the devices constantly.

By embedding the IDS in the IoT nodes, communication overhead is avoided [69]. However, it consumes the resources (processing, memory, and power) generally associated with low-energy devices [70]. This may be impractical in many instances. However, distributing IDS sensors across the IoT network on a few dedicated devices it may make this approach feasible. Still, the network architecture must be altered to allow devices to communicate with the dedicated nodes [14,21].

3. System model

This section provides our approach's system description, design, and model.

3.1. MIoT system

The MIoT system under consideration in this research article comprises the part of the patient, the components depicted in Fig. 4. Note that Fig. 4 represents a part of the system and, most specifically, the part of the components that are in the patient room. The diagram depicts a standardized MIoT (Mobile Internet of Things) system, wherein a selection of sensors are interconnected with a computer and afterward linked to the MIoT gateway via the computer. Furthermore, certain sensors are directly linked to the MIoT gateway. It should be noted that many communication protocols for the Internet of Things (IoT), including Bluetooth, Bluetooth Mesh, ZigBee, ZigBee Mesh, Lora, and LoraWAN, can be employed to establish connectivity between all devices at the gateway. A Raspberry PI was configured as an access point for the Internet of Things (MIoTs) using the IP protocol over WiFi Direct. The diagram illustrates how healthcare sensors gather various physiological data, including heart rate, blood pressure, and body temperature. These data are then transmitted via an Internet of Things (IoT) gateway device to the cloud, where they undergo processing and analysis. One potential use for this system is in the field of remote patient monitoring, wherein health metrics for a particular patient are periodically transmitted to the respective healthcare experts' devices. As discussed in Section 2.2, the MQTT protocol is a widely used application layer for collecting and transmitting healthcare sensor data. It operates on a publish-subscribe architecture. Owing to the inherent sensitivity of healthcare data, the health parameters will be subjected to encryption, rendering them accessible just through decryption by the device or system employed by the healthcare professional responsible for the patient's care. Furthermore, the classification model undergoes alterations, and both the Intrusion Detection System (IDS) and the Firewall (FL) signals will be subjected to encryption. Consequently, only the designated recipient or intended destination can decipher the encrypted messages and model modifications. The procedure mentioned above may be executed by employing public key cryptography as the encryption mechanism and utilizing the Certificate Authority server to distribute the accurate public keys of the sinkholes in conjunction with the cloud server.

3.2. Attack model

Intruding into an MIoT system is a pivotal issue, as IoT systems are resource-constrained and have a large attack surface. An MIoT system must have a protective layer to avoid malicious attacks. An intruder follows the following methodology to achieve its malicious goals against an IoT system. The following subsection will show an attacker's steps to attack an MIoT system.

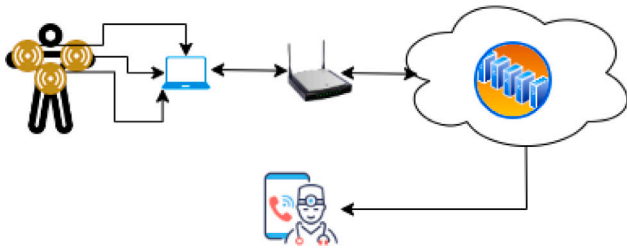


Fig. 4. MIoT system model.

Table 4

Attack methodology.

System Reconnaissance		Cyber-Attack Design
Collect Information	→	Devise Plan of action
Inspect Target		Choose correct tools
		↓
Implementation		Entry into system
Sustain attack	←	Intrude IoT system
Pave more attacks		Physical/Remote

3.2.1. System reconnaissance

This is the first stage that involves collecting data primarily about devices in the target IoT system. The attacker may require information about the hardware, IoT providers, and even crucial telemetry data that may be obtained from many sources. Another important thing is to know what services are available in that system. You can find this out by doing things like port scanning.

3.2.2. Cyber-attack design

The tools that are most well suited for the attack on such an IoT system need to be carefully curated based on information obtained from the previous stage in this stage. Depending on the attack's needs, the attacker may use a mix of these tools to make a base plan for the attack strategy.

3.2.3. Entry into the system

This is the stage at which the attacker launches the attacks using the tools selected in the previous stage, gaining access to the system by exploiting vulnerabilities in the target IoT system.

3.2.4. Implementation

The attacker takes control of the IoT system using the earlier strategy and inflicts planned damage. The attack must be sustained without losing access to the system and set the base by making it easier for future intrusions. Table 4 represents the four-stage attack methodology.

In the case of an MIoT system consisting of the components described in the previous subsection, attacks could have the purpose of making the system unavailable through power drainage of devices or denial of service (DoS) attacks, capturing/modifying sensitive health data, and accessing the system components in an authorized manner. Our focus in this work is mainly on the first-stage and third-stage attacks for intrusion detection. Below, we provide a brief description of each attack type considered. Note that network scanning (NMAP)/reconnaissance attacks come under “system reconnaissance” (they are considered attacks because they can give valuable information to the attacker), and DDoS, MitM, and Brute force authentication attacks come under the “entry into the system”.

1. **Distributed Denial of Service (DDoS) attack [71]:** DDoS attacks prevent legitimate users from accessing a system by flooding requests from multiple systems (attackers) to a single targeted system. The attacker tries to get the IP of the targeted system by using various powerful tools. When this attack is

attempted successfully, the number of requests exceeds the system's capacity. As a result, users experience a delayed response, or sometimes the system hangs abruptly.

2. **Man-in-the-Middle (MitM) attack [72]:** MitM is an attack where the attacker will be between the user's conversation and the system. The data flow seems normal, but the attacker will view the data passing to the cloud. This attack aims to sniff the data passing between the system and the cloud. The data obtained during this attack can be used in many ways. i.e., fund transfers, password exchange, intercepting data, etc.
3. **Brute Force Authentication attack:** This is a password guessing attack. The attack is an attempt to find the credentials of a legitimate user by systematically trying out all the possibilities. The possibilities depend on the length and the complexity of the password. The targets are the devices that require authentication. We can also perform a dictionary attack where we will have a list of commonly used passwords that the user/system might use.
4. **Network scanning/reconnaissance attack [73]:** A network mapper (NMAP) is a powerful tool attackers use to discover network information about a system. It helps discover hosts and services by sending packets and analyzing the responses. The IP addresses and other essential data are received from NMAP packets. Different transport layer protocols are used to send error messages, which include Transmission Control Protocol (TCP), User Datagram Protocol (UDP), Stream Control Transmission Protocol (SCTP), and Internet Control Message Protocol (ICMP). That is why, for us, they are considered attacks because they can give valuable information to the attacker, and they must be stopped from the IDS.

4. Methodology of the proposed approach

This section describes our proposed IDS overall approach for the MIoT system discussed in Section 3 along with the assumptions of the investigation. Additionally, we provide an algorithm for the MIoT Enterprise Resource Planning (ERP) system that shows the component's interruptions. This work focuses on detecting intrusions against the gateway device through the network and updating the model (for classification) using Federated Learning. Intrusions on the cloud components for reporting and data storage are outside the scope of this work and will be addressed in future work. Therefore, the intrusion detection algorithms are run at the gateway device as the sinkhole. Fig. 5 shows a high-level view of the proposed end-to-end intrusion detection approach. A three-step Intrusion Detection System (IDS) methodology is proposed. Firstly, this study proposes a machine learning (ML) approach to identify network threats using classification techniques on MIoT gateways/sinkholes. Continuing, the process involves the identification of unidentified system attacks and facilitating the learning of these attacks by the Intrusion Detection System (IDS) through training the classification model with attack records. Ultimately, the distant cloud server and other sinkholes are continuously updated using Federated Learning techniques. The cloud Enhanced Random Forest machine learning model updates the continuously trained sinkholes on the local area network (LAN). Conversely, the sinkholes update their model with a newly identified attack towards the cloud model. In a more particular manner, Federated Learning (FL) utilizes the knowledge that our classification model is an Enhanced Random Forest (also called ensemble random forest) and incorporates the variations in nodes and trees into the primary cloud ERF model as novel assaults are identified, utilizing the updated model that encompasses the modified nodes and trees. The cloud server subsequently informs the other MIoT gateways about the updated tree structure changes (we can call them Delta changes) to maintain consistency with the new model and identify the new attack. This study aims to utilize cloud-based transfer learning to enhance and update our models for specific sinkholes or all sinkholes.

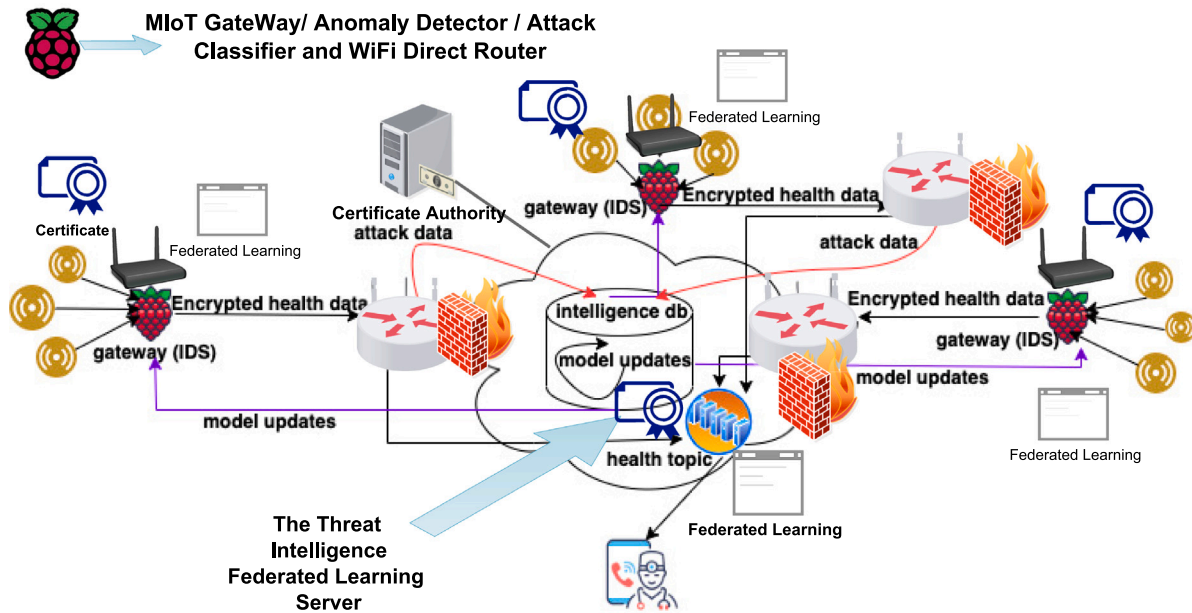


Fig. 5. Architecture of the end-to-end intrusion detection approach.

This approach will enable us to incorporate more attack scenarios and accomplish real-time model optimization. In order to enhance the precision of our model assessment, we perform our tests within our emulation environment (as described in Section 4), utilizing a cloud server that employs the currently enabled model at the sinkholes of several MIoT networks.

In accordance with the aforementioned, the primary aim of this research is to demonstrate the feasibility of classifying numerous attacks using the Random Forest approach, as indicated in Section 5. Furthermore, the classification using ERF and detection of anomalies (attacks) can be achieved in a power, memory, and CPU efficient manner, as demonstrated in Section 5 by utilizing One-Class SVM. Thirdly, federated learning (FL) has been demonstrated to be both practical and efficient, as seen in Section 5. This approach enables MIoT gateways and cloud servers to undergo continuous training in a mutually beneficial manner. For instance, a cloud server can be trained with new datasets in addition to the existing ones, creating an instantaneous model. The differences between the new and old models can be calculated, and the updated model can be shared and transferred to the network sinkholes. In this study, we conduct an evaluation of various classification techniques, namely Support Vector, Naive Bayes, K-Nearest Neighbours, Decision Tree, and Random Forest, in order to classify different types of attacks, including MitM, DDoS, Brute Force Authentication, and NMAP identification (we consider NMAP scan as an attack in our investigation). Additionally, we employ well-established machine learning techniques such as Support Vector, Naive Bayes, K-Nearest Neighbours, Decision Tree, and Enhanced Random Forest to identify unknown attacks using One-Class SVM, to achieve high accuracy in the identification process. Moreover, the most effective strategy for power, CPU, and memory allocation was the Enhanced Random Forest machine learning model for classification, with an accuracy rate of 99.8%. Furthermore, to discover anomalies belonging to a single class, we employed the One-Class Support Vector Machine (SVM) algorithm, which has been widely recognized as the most effective technique for anomaly detection, as reported in the literature [38–40]. Additionally, our findings demonstrate that the One-class Support Vector Machine (SVM) strategy for unknown attack identification is among the most effective methods for conserving power, CPU use, and memory. Furthermore, it exhibits a high level of accuracy in identification, achieving a rate of 99.7%. This paper presents a novel approach for designing a feasible MIoT IDS that considers various requirements

such as power consumption, delay constraints, and high accuracy. The proposed approach utilizes an FL green machine learning-based technique, which leverages one-class SVM for anomaly/attack detection and employs an Enhanced Random Forest model for attack classification at the sinkhole. Subsequently, the central Enhanced Random Forest model is optimized through the utilization of cloud computing, incorporating the differences in Enhanced Random Forests, such as intermediate nodes and leaves. This process also extends to updating the remaining MIoT sinkhole(s), which can be executed in a bilateral manner. In certain scenarios, the cloud model is updated first, followed by the subsequent updates to the remaining MIoT components.

The proposed intrusion detection approach consists of the main components as described below:

- *The GateWay IDS/MIoT GateWay (GW)* is responsible for running the intrusion detection algorithms on the network flows resulting from the connections established with the cloud and sending anomalous data (the delta of classification model) to the cloud threat intelligence server.
- *The Attack Classifier* is an ML model for determining the classification of network flows, including their attack classes, which run in the gateway IDS.
- *The Anomaly/Identification Detector* is an ML model for identifying/detecting anomalies in received network flows, which runs in the gateway IDS.
- *The Threat Intelligence Federated Learning Server/The cloud server model* acts as a central point of intelligence collection for known/unknown attack information of the system admins (via MQTT). It gathers the classification model deltas generated from the IDS gateways that are calculated based on anomalous data, and it updates its own local classification model for the cloud IDS. It also communicates with the rest of the gateway IDSs connected to it to update their attack classification model with newly discovered attack data using the provided delta. Note that the intelligence server digitally signs the deltas (calculated from the updated model) before being sent to the rest of the gateway IDSs.

Upon receipt of any network flow at the gateway, the following steps are taken to detect intrusions (as shown in Fig. 6):

1. The network traffic that has been received is analyzed in order to extract the necessary features for the machine learning model.

It should be noted that the process of anomaly identification and attack classification occurs not at the PCAP files themselves but rather at the converted PCAP files that have been transformed into CSV feature data. Therefore, within the gateway Intrusion Detection System (IDS), a service consistently utilizes the “CICFlowMeter tool” to transform Wireshark PCAP files into feature records.

2. The extracted features are input to the anomaly detector for determining whether the traffic is normal or an instance of a known anomaly.

- (a) The attack classification model is run to the network flow to determine the attack's class.
- (b) If the network flow is classified as an attack, it informs the Threat Intelligence Federated Learning Server

1. The system employs encryption techniques to secure the data intended for transmission to the intelligence database, which will be used for future IT-related information regarding past, current, and potential attacks. Before encryption, the system selectively excludes identifying records associated with private data, such as the Internet Protocol (IP) addresses of user devices, to safeguard user privacy. These excluded records are then forwarded to the Threat Intelligence Federated Learning Server.

- (c) Alternatively, if the network flow is not categorized as an attack, and in the case where the network flow is classified as an anomaly and an unknown attack, the approach should proceed to update its classifying model. This update is achieved through the utilization of federated learning, wherein the differences in the Enhanced Random Forests are transmitted to the remaining models, as depicted in Fig. 7. More specifically, our proposed approach is executing the following:

1. It identifies the related records to the anomaly and associated features. Then, the system trains the classification model used for anomaly detection with the new records according to the executed feature selection.
2. Compare the resulting model of the attack classification with the old model (saved in the disk of the MIoT gateway) and calculate the differences in the model. Also, overwrite the old model with the current model on the disk.
3. Send the calculated differences between the old model that is saved in the disk of the MIoT gateway and the running model to the cloud model.
4. The cloud server model (cloud threat intelligence server) is updated with the new changes and replaces its old model at the disk with the new model.
5. The cloud server model (cloud threat intelligence server) sends the differences of the model to the rest of the MIoT.
6. The MIoT gateway receives the model differences by updating its model with the new changes, replacing its old model at the disk with the new model.
7. The unknown attack identified IoT gateway removes private data from the flow to preserve user privacy. Thus, from the identifying records of the features that are related to the private data, the IoT gateway removes private information (e.g., Internet Protocol (IPs) addresses of user devices). Afterward, it sends the flow records to the Threat Intelligence Federated Learning Server.

- (d) If an anomaly or attack was detected in the previous step, the IDS raises the alarm and takes appropriate mitigation action.

4.1. ERP-based intrusion detection

The ERP algorithm 1 that orchestrates our approach's Enterprise Resource Planning system integrates various MIoT network components to enhance its security framework. Utilizing Federated Learning, the ERP system consolidates threat intelligence and coordinates between MIoT gateways, anomaly detection systems, attack classifiers, and a central threat intelligence server.

Algorithm 1: ERP algorithm for the Federated Learning-based IDS for MIoT Network Security Hardening

Input: Network traffic flows from MIoT Gateways
 Current classification and anomaly detection models
 Cloud threat intelligence database (Intelligence db)
 Federated Learning Server (FL Server)
Output: Updated classification and anomaly detection models
 Alarms for identified threats
 Encrypted and privacy-ensured threat data for Intelligence db

// Feature Extraction:
 Convert incoming network traffic (PCAP) to feature records (CSV) using CICFlowMeter
 Extract necessary features for the machine learning models

// Anomaly Detection and Attack Classification:
 Input extracted features to the anomaly detector
if traffic is normal **then**
 | **return**
else
 | Classify the traffic flow using the attack classification model

// Model Update and Threat Intelligence Sharing:
if flow is classified as an attack **then**
 | Encrypt and transmit attack data to the FL Server
else
 | Train the classification model with new records
 | Compare the new model to the old model and calculate deltas
 | Update the model on the MIoT Gateway
 | Transmit model deltas to the FL Server
 | FL Server updates its model and transmits deltas to other MIoT Gateways
 | Other MIoT Gateways update their models with new deltas

// Response and Mitigation:
if an anomaly or attack is detected **then**
 | IDS raises an alarm
 | Perform mitigation actions
else
 | // Continue monitoring

Consequently, utilizing a threat intelligence server facilitates the ongoing update of machine learning models at the gateway intrusion detection system (IDS) by incorporating data obtained from various gateways. This process is particularly relevant in the context of anomaly detection, as it enables the linked IDS to remain informed about emerging assaults and then implement suitable mitigation measures.

It should be noted that the data transmitted between the MIoT gateways and the threat intelligence server is subject to encryption and digital signing. This process involves the utilization of pre-assigned digital signatures, which were allocated to the devices during their configuration by the Certificate Authority (CA).

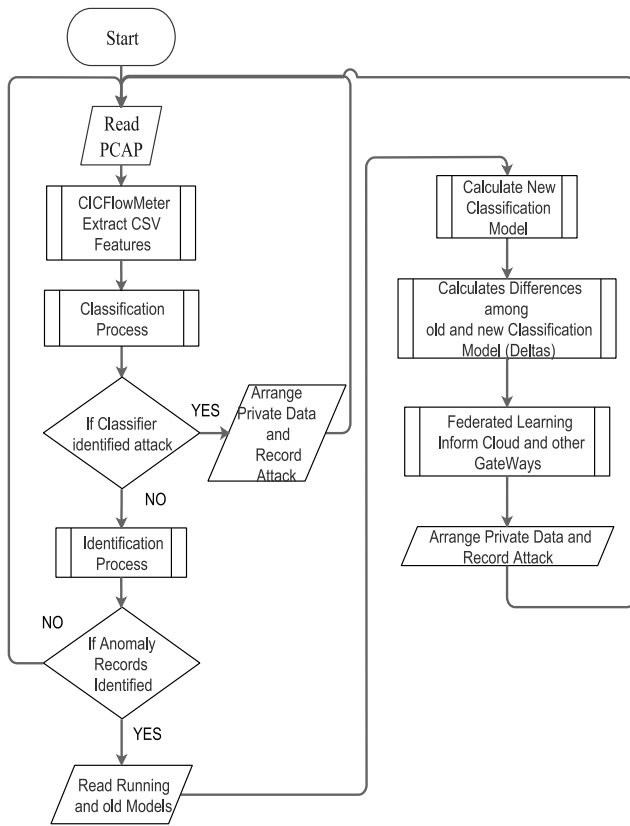


Fig. 6. The flowchart of the proposed algorithm.

The proposed IDS approach is secure against various attacks listed below that could be launched against it.

- *Model poisoning attacks*, which may be launched by malicious gateways attempting to provide falsified data to the central intelligence server, are thwarted using digital signatures (shared through a Certified Authority) on the attack data being transmitted. Any malicious gateways can be identified through their signatures and subsequently disconnected by the server.
- *Rogue intelligence servers* that might send tainted models to the gateways are prevented by using digital signatures on the models they transmit. Gateways verify these signatures before integrating the models into their Intrusion Detection Systems (IDS).
- *Denial of Service (DoS) and power drainage attacks*, aimed at reducing the availability of the gateway, are detected using DoS/DDoS detection algorithms running within the IDS's attack classifier.
- *Data leakage attacks*, which intend to steal privacy-sensitive data from the gateways, are thwarted by obfuscating sensitive feature values like IP addresses before transmitting attack data to the cloud. Health-related data are sent separately in encrypted form to the relevant server. Furthermore, any changes to the model (for classification) and the IDS and Federated messages are transmitted separately in encrypted form to the corresponding cloud server or sinkholes.

4.2. Assumptions of the proposed approach

In this section, we provide the assumptions of the investigated examination. The assumptions of our investigation are the following:

- This examination does not investigate the proposed IDS's reporting methods using the MQTT protocol; this will be a future focus of the investigation.

- This analysis does not investigate the security among the suggested IDS components, and security is not quarantined; this will be a future focus of the investigation. For the current investigation, security is forced via Public Key encryption mechanisms and the Certificate Authority.
- The data storage handling in terms of the type of storage needed and speed of storage (Serial Advanced Technology Attachment (SATA), Solid State Drives (SSD)) will be discussed in a future work of this investigation.

4.3. Algorithm of the proposed approach

We demonstrate the proposed approach steps in Alg. 2. This algorithm highlights the novelty and contributions of the proposed investigation.

Algorithm 2: GEMLIDS-MIOT Algorithm

Result: Keeping the MIoT network secure
RUNNING_MODEL: The running model used by the classification algorithm, saved in memory
PCAP: The PCAP generated files per second
CICFlowMeter: A method that converts the PCAP files to feature records (as shown in Section 5.1)
FeaturesRecords: The converted feature records for model training and anomaly detection
RecodsIdentified: The feature records identified with anomalies
ANOMALY_DETECTED: The anomaly detector that identifies an anomaly and returns the RecodsIdentified using the ML approach (as shown in Section 4.4.2)
FederatedLearningExecution: The FL algorithm (as shown in Section 5.3)
AttackClassificationML: The attack classification ML approaches (as shown in Section 4.4)
ClassificationResult: The classification result from the ML approaches; it is a numerical result that relates each category to a number, where zero indicates no classification

```

while true do
    FeaturesRecords = CICFlowMeter(PCAP);
    ClassificationResult =
        AttackClassificationML(FeaturesRecords);
    if (count(ClassificationResult) ≥ 1) then
        | Inform the Central System;
    else
        RecodsIdentified =
            ANOMALY_DETECTED(FeaturesRecords);
        if (count(RecodsIdentified) ≥ 1) then
            | FederatedLearningExecution(RecodsIdentified,
            | SAVED_MODEL, RUNNING_MODEL);
            | Inform the Central System;
        end
    end
end
end
  
```

4.4. Machine learning approaches used in the MIoT investigation based intrusion detection

This section provided the insides of the proposed ML models on how to optimize our models to achieve better accuracy.

The proposed IDS utilizes lightweight, supervised ML models for identifying and detecting intrusions through anomaly detection and classification techniques. This analysis is performed on data acquired from network flows, which are also depicted in this section. The following paragraph overviews the utilized network flow features and the used ML models.

A supervised learning approach is an approach that takes a certain amount of labeled training data to train models, which is very good at

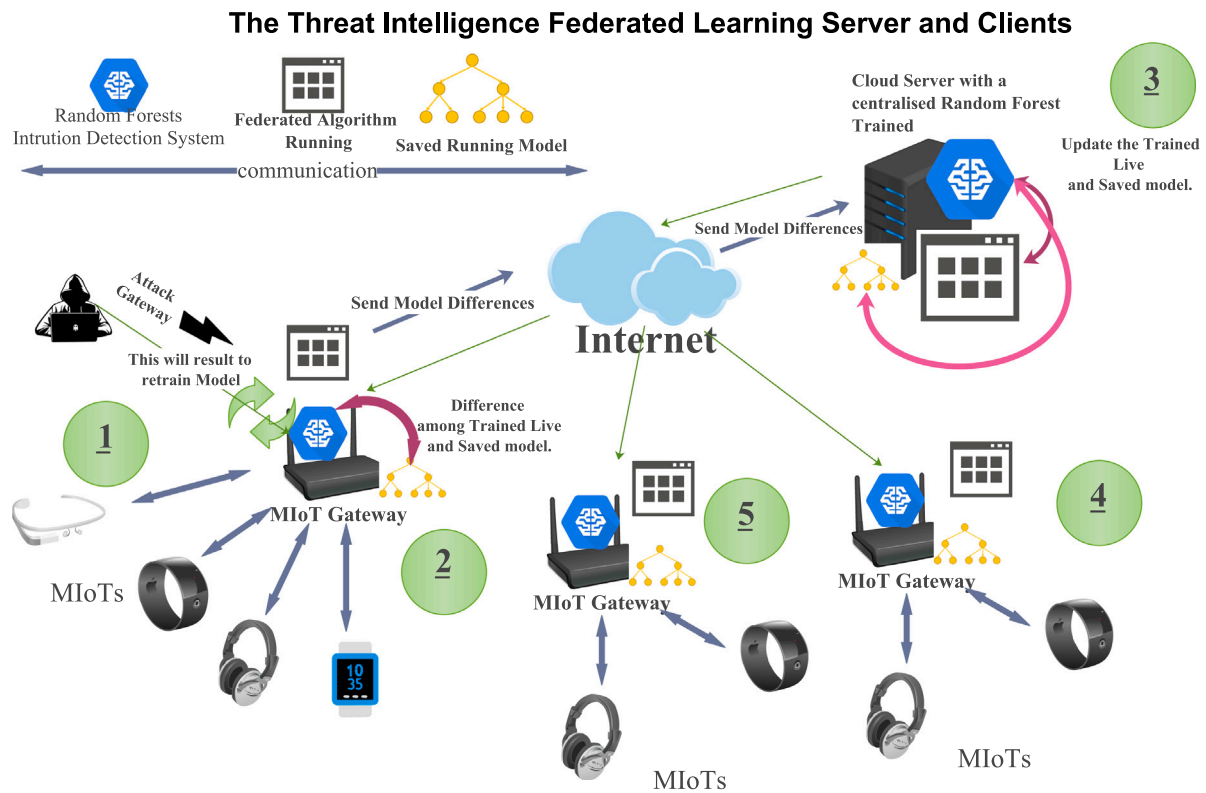


Fig. 7. Identification of an anomaly and the use of federated learning.

classification problems, therefore making it reasonable for us to take the same approach. A well-annotated dataset is crucial in training a good model. The dataset's quality depends on the proper labeling done by humans, essentially making it a "Human-in-the-loop" process in the identification and classification ML model training process. Fortunately, this step will produce well-defined findings due to the precise connection information returned by the testing phase. Once this has been accomplished, the models can be trained, causing the selected algorithms to learn and "teach" themselves to spot patterns in a dataset.

4.4.1. Network features

The dataset we utilized in this work has 80 features obtained from network flows using the CICFlowMeter tool. After feature selection based on importance scores, we found the best 25 features to train our models, as explained in Section 5. The best features that were derived from the set of 80 features with their descriptions are listed in Table 5.

4.4.2. Examined ML models for classification and identification (anomaly detection)

This section provides the algorithms used for traffic identification and classification by the gateway IDS used in the first stage. Also, it utilizes only one ML approach for anomaly detection due to its high accuracy and investigation in the open literature. It is used in the second stage for unknown attacks. It examines multiple competitive approaches for classification to select the most accurate approach at the end. So, our investigation is exploring the different ML classifiers² for the classification and identification of each attack.

² The objective of a classification problem is to assign new objects to predetermined classes based on data provided by known objects, their classes, and their attributes, as well as data about their classes. Candidates for a solution have been chosen in consideration of evaluation criteria such as performance (equivalent to the quality of outcomes), complexity, and inference time. Using the following algorithms, we can fit models to training data gathered from earlier steps, and our experiment will continue.

We will utilize the following ML approach for the anomaly detection problem because it is widely used and demonstrated in the open literature to be the most effective method for this purpose [38–40]. The ML method we use is shown in Section 2.2.5 and is the One-Class SVM approach.

For our classification problem, we will compare the following approaches (as shown in Section 2.2.5):

- Support Vector Machines (SVM).
- K-Nearest Neighbors (KNN).
- Decision Tree.
- Random Forest.
- Naïve Bayes.

4.4.3. ML approach training and testing

A portion of the available data, commonly called the training data, is used to construct the model. The training dataset is used to assess the performance of the dataset. Typically, it is computed as a proportion of the total dataset, expressed as a percentage. Ensuring that the training set is entirely distinct from the data employed for training purposes is imperative. They determine whether the model acquired or memorized knowledge from the input is challenging. Overfitting is a frequently encountered phenomenon. To mitigate this issue, it is necessary to ensure that the test set is distinct. Utilizing an excessively tailored model has the potential to provide exceedingly poor outcomes. Numerous libraries offer mechanisms for partitioning datasets into training and test data subsets. Occasionally, a need to rearrange the order of data from its present arrangement may arise. Typically, a designated proportion, such as 30%, is allocated for the purpose of testing, while the remaining portion is allocated for training. Conducting tests on the designated test set will yield prediction outcomes, enabling algorithmic performance evaluation. In this investigation, 30% of the data is for training, and the results are examined using k-fold validation, as shown in Section 5.2.1.

Table 5
Network features.

<i>flow_bytes_s</i>	Number of flow bytes per second
<i>flow_pkts_s</i>	Number of flow packets per second
<i>fwd_pkts_s</i>	Number of forward packets per second
<i>bwd_pkts_s</i>	Number of backward packets per second
<i>fwd_pkt_len_max</i>	Maximum size of packets in forward direction
<i>bwd_pkt_len_max</i>	Maximum size of packets in backward direction
<i>fwd_pkt_len_mean</i>	Mean size of packets in forward direction
<i>fwd_pkt_len_std</i>	Standard deviation size of the packet in forward direction
<i>bwd_pkt_len_max</i>	Maximum size of the packet in backward direction
<i>bwd_pkt_len_mean</i>	Mean size of the packet in backward direction
<i>bwd_pkt_len_std</i>	Standard deviation size of the packet in backward direction
<i>pkt_len_max</i>	Maximum length of a packet
<i>pkt_len_mean</i>	Mean length of a packet
<i>pkt_len_std</i>	Standard deviation length of a packet
<i>pkt_len_var</i>	Variance length of a packet
<i>fwd_iat_tot</i>	Total time between two packets sent in the forward direction
<i>bwd_iat_tot</i>	Total time between two packets sent in the backward direction
<i>pkt_size_avg</i>	Average size of packet
<i>init_fwd_win_bytes</i>	The total number of bytes sent in the initial window in the forward direction
<i>init_bwd_win_bytes</i>	The total number of bytes sent in the initial window in the backward direction
<i>fwd_bytes_b_avg</i>	Average number of bytes bulk rate in the forward direction
<i>fwd_pkts_b_avg</i>	Average number of packets bulk rate in the forward direction
<i>bwd_bytes_b_avg</i>	Average number of bytes bulk rate in the backward direction
<i>fwd_seg_size_avg</i>	Average size observed in the forward direction
<i>bwd_seg_size_avg</i>	Average size observed in the backward direction

4.5. Proposed federated learning approach

This section contains the implementation of the FL. It provides the FL parameters that are set in our emulation system, the event that forces the FL to start and inform the network, the algorithm for calculating the delta that is sent to the network, and finally, the algorithms that should run on the gateways and the cloud for the FL to achieve its goal of informing all the models in the MIoT network. As shown below, in our approach, FL will be implemented using DFS (as shown in Section 2.2.6) on Enhanced Random Forests³ for calculation of Delta differences among the old with the new classification model.

One of the primary concerns with federated learning is the exposure to potential risks such as backdoor injections or malicious manipulation of the training data [74–76]. Our approach tackles this issue by combining digital signatures and robust encryption techniques. Specifically, digital signatures are used to authenticate the integrity and origin of the data shared across the network. This ensures that unauthorized entities have not altered or tampered with training data. Additionally, encryption is applied to the training data while it is being stored locally and during transmission to the federated learning server. This dual-layered security mechanism effectively shields the system against attempts to inject backdoors or manipulate the training dataset, thereby maintaining the overall integrity and reliability of the federated learning process.

4.5.1. Federated learning parameters

In this section, we show the values of different parameters that optimize learning to control the FL process as shown in Table 6.

4.5.2. Event that triggers the federated learning algorithm

Every sinkhole is trained using an Enhanced Random Forest algorithm, which has been identified as the most accurate machine learning model with the least computational power and memory capacity, as evidenced in Section 5. During the training process, the sinkhole retains a model that is stored in the cloud or a centralized machine learning model. This stored model is used as a reference to detect any modifications or updates that need to be made to the present training

Table 6
Federated learning parameters.

Parameter	Value
Number of rounds	$R = 1$
Total number of nodes	$N = 12$
Fraction of nodes per iteration	$FN = 100\%$
Local batch size	$BS = 10 \text{ KB}$
Local training iterations	$I = 1$
Local learning rate	$\eta = 100\%$

model. Furthermore, the system initiates a request for the most recent version of the model through a Representational State Transfer (REST) protocol from the cloud-hosted machine learning (ML) model. This request is made in an encrypted manner, ensuring the security and integrity of the data. A digital signature accompanies it to verify the authenticity of any transmitted content. As outlined in Section 4, once the anomaly detection process identifies an anomaly using the anomaly detector, which often represents an unknown assault/attack, the system generates a new model. This is achieved by training the existing model with the inclusion of the extra records.

4.5.3. Calculation of model differences in random forest

This section presents the algorithm employed for calculating the *Delta* of the running model compared to the saved model at any MIoT Gateway. The algorithm in question incorporates specific components derived from the widely recognized depth-first search algorithm (DFS). The Depth-First Search (DFS) technique is utilized for tree traversal (for further details on the DFS, please refer to Section 2.2.6). This is because the Enhanced Random Forests classifier is derived from the Decision Tree classifier and comprises many individual decision trees. Therefore, the depth-first search (DFS) methodology will involve sequentially accessing the decision trees stored in an array implementation, starting from the leftmost tree in the random forest. Each tree will be assigned an incremental index value, such as “0” for the first tree, “1” for the second tree, “2” for the third tree, and so on. The modified depth-first search algorithm will also include a distinct data structure, specifically a list, to store the visited edges as a set. To do this, the recursive procedure must receive the parent vertices of the vertex being inspected as an argument. Each element in the array will consist of a set that contains a representation of the additional edges. Each edge is composed of two vertices that are connected by the edge. In greater detail, the edge possesses distinct attributes, including the

³ We have demonstrated that they are one of the most precise machine learning models, with a high level of efficiency in terms of reserving power, CPU, and memory.

vertex it is connected to, its special feature, and its corresponding weight. Upon traversing all Decision Trees within a Random Forest, we proceed to evaluate the differences in terms of edges between each individual decision tree and the previously saved traversal list of the old model. This comparison allows us to identify any edges that have been added or removed. The symbol Δ denotes an array consisting of decision trees. Each decision tree within the array is represented by a set of edges, reflecting the differences specific to that decision tree. The provided Algorithm 3, demonstrates the modified Depth-First Search (DFS) algorithm, which includes the computation of the discrepancies across tree models. Fig. 8 displays a representation of the Decision Trees for both the old and new models, derived from the depicted decision trees and their disparities. The computation results obtained from executing the technique outlined above are depicted in Fig. 9 (as shown in Fig. 8 and Fig. 9).

Algorithm 3: Deltas Calculation Algorithm Based on DFS

Result: DFS
SAVED_MODEL: The model currently used by the Classification ML approach, saved on the disk
RUNNING_MODEL: The running model used by the classification algorithm, saved in memory
Delta: The models' differences, an array holding the differences of each model based on each decision tree
DecisionTreeSM, **DecisionTreeRM**: The decision trees of SM and RM
visitedVertex: An array used for visited vertices
fatherVertex: The father of the investigated vertex, used to get the edge
investigatedVertex: The investigated vertex
investigatedEdge: The investigated edge
setOfEdges: The set of edges
checkVisitedVertex: The function that checks if the vertex is visited
getInvestigatedEdge: Get the associated edge between *fatherVertex* and *investigatedVertex*
getInvestigatedVertexChildren: Get the children of *investigatedVertex*
children: The children of *investigatedVertex*
Function AdaptedDFS(*fatherVertex*, *investigatedVertex*):
 if (*checkVisitedVertex*(*investigatedVertex*) == 0) **then**
 Add *investigatedVertex* to *visitedVertex*;
 investigatedEdge = *getInvestigatedEdge*(*fatherVertex*, *investigatedVertex*);
 if (*count*(*investigatedEdge*) == 1) **then**
 | Add *investigatedEdge* to *setOfEdges*;
 end
 children = *getInvestigatedVertexChildren*(*investigatedVertex*);
 while (*count*(*children*) > 0) **do**
 | Remove a child from *children*;
 | AdaptedDFS(*investigatedVertex*, *child*);
 end
 else
 return;
 end
 ;
Function Main(*SAVED_MODEL*, *RUNNING_MODEL*):
 Initialization;
count = 0;
while (*count*(*RUNNING_MODEL*) > 0) **do**
 Read *DecisionTreeSM* and *DecisionTreeRM*;
 Delta[*count*] = AdaptedDFS(*DecisionTreeRM*.*ROOT*, *nothing*)
 - AdaptedDFS(*DecisionTreeSM*.*ROOT*, *nothing*);
 count++;
end
return

4.5.4. Federated learning algorithms

This section presents the FL Algorithms 4 and 5 that are required to be executed at the MIoT's gateway and the cloud ML model server, respectively, to ensure efficient and dependable execution of the model's updates during the FL process. To provide the necessary atomicity of

the federated learning (FL) process execution, the algorithm employs a Queue to handle the requested changes to the enhanced Random Forest model. This approach aims to facilitate the successful and sequential updates of the model for both the cloud server and all other models within the network. The delta computation will be demonstrated in Section 4.5.3.

Algorithm 4: Federation Learning Algorithm Run in Each MIoT Gateway

Result: Updates Cloud ML Model
SAVED_MODEL: The model currently used by the Classification ML approach, saved on the disk
RUNNING_MODEL: The running model used by the classification algorithm, saved in memory
PCAP: The PCAP generated files per second
CICFlowMeter: The method that converts the PCAP files to features records shown at 5.1
FeaturesRecords: The converted features records for model training and anomaly detection
RecodsIdentified: The features records identified with anomaly
saveModeltoDisk: The calculation of Model Differences Algorithm shown at Section 4.5.3
ANOMALY_DETECTED: The anomaly detector that identifies an anomaly and returns the *RecodsIdentified*
CalculateModelsDifference: The calculation of Model Differences Algorithm shown at Section 4.5.3
Delta: The model differences
Cloud_ML_Model_Server_IP: The IP address of the cloud ML Model Server
 Initialization ;
while true do
 FeaturesRecords = *CICFlowMeter*(*PCAP*);
 RecodsIdentified =
 ANOMALY_DETECTED(*FeaturesRecords*);
 if (*count*(*RecodsIdentified*) ≥ 1) **then**
 Delta = *CalculateModelsDifference*
 (*RecodsIdentified*, *SAVED_MODEL*,
 RUNNING_MODEL);
 SAVED_MODEL = *RUNNING_MODEL*;
 Send the **Delta** to the *Cloud_ML_Model_Server_IP*
 encrypted;
 end
end

5. Experimental evaluation

This section presents the outcomes of experiments conducted on a real system configuration utilizing heart rate sensors and a Raspberry Pi 3B+ device. This experimental setup has been chosen to replicate real-world conditions in MIoT environments closely. The focus of these experiments is not only on identifying mimicked threats but also on rigorously evaluating the effectiveness of different machine-learning models through comprehensive performance testing. This includes both quantitative and qualitative analyses. Quantitatively, we assess the models' performance using metrics such as accuracy, precision, recall, and F1-score. These metrics objectively measure how well our models can detect and classify various types of cyberattacks. Qualitatively, this research investigates our models' practical applicability and robustness, offering insights into their real-world effectiveness and limitations. This holistic approach to evaluation ensures a thorough understanding of the models' capabilities and contributes significantly to the field of MIoT security. Even more, in this section, we dive deeper into the specific aspects of our study: MIoT Attack Dataset, Experiments Results, and Discussion. This includes the Attack Classification Results Used to Select the ML Approach for the First Stage of our Investigation,

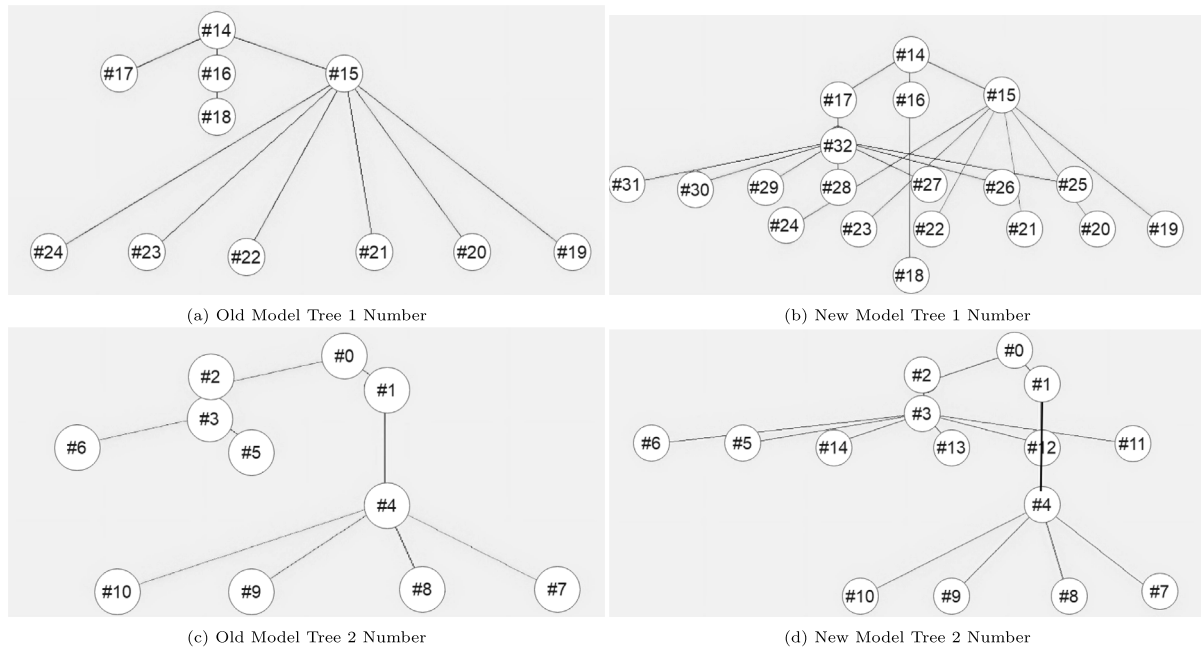


Fig. 8. Changes of enhanced Random Forest in decision trees after a new attack identification.

```

run:
Finished
The DFS result for the Tree tree_1:0,2,3,6,5,1,4,10,9,8,7,
The DFS result for the Tree tree_1_new:0,2,3,6,5,14,13,12,11,1,4,10,9,8,7,
Differences among tree_1 and tree_1_new:
Edge (3 3.0 11)
Edge (3 2.0 13)
Edge (3 8.0 14)
Edge (3 8.0 12)
Finished
The DFS result for the Tree tree_2:14,17,16,18,15,24,23,22,21,20,19,
The DFS result for the Tree tree_2_new:14,17,32,31,30,29,28,27,26,25,16,18,15,24,23,22,21,20,19,
Differences among tree_2 and tree_2_new:
Edge (17 32.0 32)
Edge (32 17.0 31)
Edge (32 15.0 29)
Edge (32 20.0 25)
Edge (32 2.0 30)
Edge (32 44.0 27)
Edge (32 18.0 26)
Edge (32 37.0 28)

```

Fig. 9. The results by running the Deltas calculation algorithm.

Anomaly Detection Results for Unknown Attacks Used in the Second Stage of the Approach, Resource Consumption and Execution Time, and Evaluation of Federated Learning Realization Performance. Each of these segments is critical to establishing the effectiveness and efficiency of our proposed system in real-world MIoT environments, providing a comprehensive view of our research's impact on the field of cybersecurity in the MIoT context. Finally, we show the time complexity regarding Big-O notation of the complete solution.

5.1. MIoT attack dataset

Fig. 10 shows an overview of the system environment we set up for generating our dataset.

IoT Environment: For our experiments, we created a health monitoring IoT system that sends crucial health vitals to the cloud, collected

with a Max30100 heart rate sensor. A Raspberry Pi 3b+ was used as a central gateway, and an ESP8266 WiFi module sent the data to the cloud.

CICFlowMeter is a tool to generate and analyze network traffic flow. It generates flows bi-directionally. i.e., forward and backward. It converts the network flows into features such as duration, number of packets, number of bytes, length of packets, sub-flow packets, push flags, etc. Along with the traffic flow features, the output has Flow ID, Source IP, Destination IP, Source Port, Destination Port, and Protocol. This paper's captured packets from Wireshark were converted into ML features using CICFlowMeter.

5.1.1. Feature analysis & feature selection for training of the investigated ML approaches

Univariate feature selection works by selecting stylish features based on univariate statistical tests. SelectKBest approach is the one

Algorithm 5: Federation Learning Algorithm Run in Cloud ML Model Server

Result: Federation Learning Algorithm Run in Cloud ML Model Server

SAVED_MODEL: The model currently used by the Classification ML approach, saved on the disk

RUNNING_MODEL: The running model used by the classification algorithm, saved in memory

Delta: The model differences

Queue_MIoT_GW_Delta_Received_IP: The Queue with the IP addresses and their Deltas (as a string in a comma-separated way) of the MIoT gateways that have received their model differences

MIoT_GW_IPs: The list of IP addresses of the MIoT gateways

Initialization ;

while true do

if (count(Queue_MIoT_GW_Delta_Received_IP) ≥ 1) **then**
 while
 ((count(Queue_MIoT_GW_Delta_Received_IP) ≥ 0))
 MIoT_GW_Received = Remove from Queue
 Queue_MIoT_GW_Delta_Received_IP the first element;
 MIoT_GW_Received_IP = Separate from the MIoT_GW_Received the IP to send Delta;
 Delta = Separate from the MIoT_GW_Received the Delta;
 RUNNING_MODEL = RUNNING_MODEL modified with the decrypted Delta;
 SAVED_MODEL = RUNNING_MODEL;
 MIoT_GW_IPs_to_Send: The list of IP addresses of the MIoT gateways without the MIoT_GW_Received_IP IP
 Send the **Delta** to the MIoT_GW_IPs_to_Send encrypted;

end

end

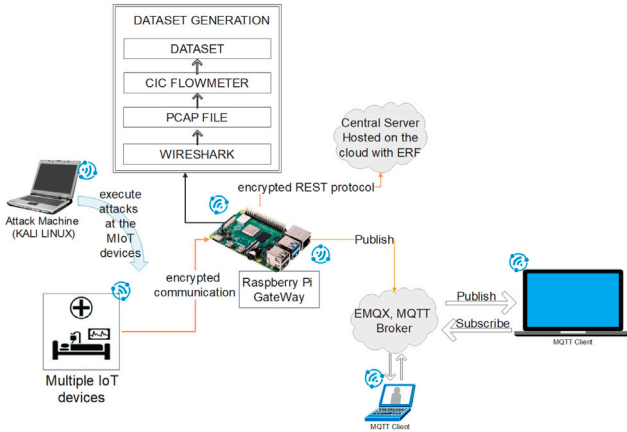


Fig. 10. MIoT attack dataset system environment.

that removes all attributes but the high-scoring features in the dataset. A chi-square test is used in statistics to test the independence of two events and is denoted by χ^2 . For example, we can obtain the observed count O and anticipated count E from the given data of two variables. Chi-Square calculates the difference between the expected count E and the experimental count O . The Chi-square formula is (Formula (7)):

$$\chi^2 = \frac{\sum(O_i - E_i)}{E_i} \quad (7)$$

where O_i = observed value (factual value) E_i = anticipated value.

Note that the chi-squared test is most commonly used in scenarios where both variables are categorical. However, it is also important to note that the chi-squared test can be applied in cases with continuous predictors, especially when these predictors are discretized or categorized before the test. This research adapts the discretizing of the continuous predictors to fit the categorical framework required for the chi-squared test. This methodological choice was made considering the specific nature of our data and the analytical objectives that this research aimed to achieve [77–79].

In feature selection, we select the features primarily affecting the response. When two features are independent, the observed count is close to the anticipated count; therefore, we will have a lower Chi-Square value. The high Chi-Square value indicates that the thesis of independence is incorrect. The selected best features are shown in Table 5 and Section 4.4.1. It should be noted that there are a total of 25 selected features selected, with a total of 21,475 elements. This includes 19,208 training elements and 2267 total training elements, as indicated in Table 7.

5.1.2. Balancing the dataset

Balancing the dataset is an important concept to our research because the IoT/MIoT IDS types of classification models are frequently encountered with an imbalanced dataset problem, where the number of the maturity class is much more significant than the nonage class. Therefore, the model faces a severe problem in training the nonage classes well. One popular approach to overcoming that weakness is to induce new exemplifications synthesized from the living nonage class. As a result, to balance the data, the SMOTE-Tomek Links system was used, where this perpetration combines the oversampling approach from SMOTE and the under-slice approach from Tomek Links [80]. SMOTE is one of the most popular oversampling methods developed by Chawla et al. [81]. Unlike arbitrary oversampling that only duplicates some arbitrary exemplifications from the nonage class, SMOTE generates models grounded on the distance of each data point (generally using Euclidean distance) and the nonage class's nearest neighbors, so the generated exemplifications are different from the original nonage class. The process for inducing the synthetic samples is as follows:

- Choose arbitrary data from the nonage class.
- Calculate the Euclidean distance between the arbitrary data and its k nearest neighbors.
- Multiply the difference with an arbitrary number between 0 and 1 and add the result to the nonage class as a synthetic sample.
- Repeat the procedure until the asked proportion of the nonage class is met.
- This system is effective because the generated synthetic data are close to the feature space of the nonage class. We have five classes in total, and the instances of each class are listed in the Table 7.

5.2. Experiments results and discussion

This section presents the outcomes of the ML model trials and a commentary on them. Also, experimentation with various ML classifiers is provided, and a performance comparison of the ML models is shown using different metrics.

5.2.1. K-Fold cross-validation

The K -fold cross-validation technique is employed to evaluate and assess the performance of the researched models. The K -fold cross-validation technique was employed with a value of $k = 5$ in order to guarantee that the model exhibited good generalization and yielded consistent metrics across various subsets of data points. The process of K -fold cross-validation involves partitioning the dataset into K equally sized folds. Subsequently, the initial $(k - 1)$ folds are employed for training the model, and the mean accuracy is computed. The k th fold is utilized to evaluate the performance of the obtained model. The algorithm followed is shown in the following list [82]:

Table 7
Dataset statistics.

Classes	Training instances before oversampling	Training instances after oversampling	Testing instances
Normal	4807	3842	965
MitM	190	3842	41
DDoS	3998	3842	791
Brute force authentication	206	3841	36
NMAP	2133	3841	434
Total	11 334	19 208	2267

1. Split the dataset into K equal-sized subsets, where K is the desired number of folds for cross-validation.
2. For each fold, do the following:
 - (a) Use K-1 subsets for training the model.
 - (b) Use the remaining 1 subset for testing the model.
 - (c) Evaluate the model's performance on the test subset using a chosen evaluation metric (e.g., accuracy, mean squared error).
 - (d) Record the evaluation result for this fold.
3. Repeat step 2 for all K folds.
4. Calculate the average performance metric across all K folds to more robustly assess the model's performance.

This algorithm outlines the basic steps of K-Fold Cross-Validation, which is commonly used to assess the performance and generalization of machine learning models.

5.2.2. Performance metrics

To assess the performance of our IDS classification models, we used multiple metrics, as detailed below. The dataset containing 80 network features and nearly 7700 packets was segmented into training and testing data. Segmentation was performed using a train-test split and was repeated using stratified K-Fold algorithms. Then, the models were created using the training data using the top five algorithms like Random Forest, Support Vector Classifier, K-Neighbours Classifier, Decision Tree Classifier, and Gaussian Naïve Bayes Classifier, and we tried making predictions on the test dataset.

We used widely used metrics for any intrusion detection system: *F1 Score*, *Precision (P)*, *Recall (R)*, *True Positive Rate (TPR)*, *False Positive Rate (FPR)*, and *False Negative Rate (FNR)*. The confusion matrix and its related metrics must be defined to understand how we calculated the above metrics.

Confusion Matrix: A tabulated representation of the results that can be used to validate the performance of trained ML models. Considering a binary classification problem, the possible outcome related to the expected outcome can be categorized as:

- **True Positives (TP):** Classified Positive (Yes) and correctly classified. TP is the number of data samples with attacks detected correctly.
- **True Negatives (TN):** Classified Negative (No) and correctly classified. TN is the number of benign samples detected correctly.
- **False Positives (FP):** Classified Positive (Yes) and wrongly classified. FP is the number of data samples with false attack detection.
- **False Negatives (FN):** Classified Negative (No) and wrongly classified. FN is the number of attack samples missed.

Given the definitions of TP, TN, FP, FN, we can compute *F1 Score* $= \frac{2 \cdot P \cdot R}{P + R}$, *Precision* $= \frac{TP}{TP + FP}$, *Recall* $= 1 - FNR$ where $FNR = \frac{FN}{TP + FN}$, and *FPR* $= \frac{FP}{FP + TN}$. Note that *Recall* can also be given by $\frac{TP}{TP + FN}$.

For any intrusion detection system, it is ideal not to miss any attacks, i.e., $FNR \approx 0$ and $Recall \approx 1$. It should also not trigger false alarms, i.e., $precision \approx 1$ and $FPR \approx 0$. Achieving both cases yields an *F1 Score* of 1.

AUC – ROC Curve: The Receiver Operator Characteristic (ROC) is a probability curve plotted with a true-positive rate against a false-positive rate. The ability to tell one class from another is measured by the Area Under Curve (AUC).

5.2.3. Cross-validation of the hyperparameters in the ML models

Hyperparameter tuning of parameters has been performed to build machine-learning models. Hyperparameter tuning is a procedure to tune the parameters that help to increase the ML model's accuracy. Two approaches are widely used to improve the ML model's performance and reduce the error rate: GridSearchCV and RandomizedSearchCV. These approaches are used to find which ML model performs the best among other ML models.

GridSearchCV [83]: GridSearchCV is one of the approaches that use cross-validation to tune the hyperparameters of the ML model. It is performed with the help of predefining the parameters in the form of a dictionary. It tries out all the combinations in the dictionary and finds the best parameters that improve the performance of the ML model.

RandomizedSearchCV [84]: RandomizedSearchCV is an alternative to GridSearchCV because it is computationally extortionate and uses all the hyperparameters to tune the ML model. RandomizedSearchCV uses a fixed number of hyperparameters to sample the ML model. This method does not give a list of specific values for each hyperparameter. Instead, it uses a statistical distribution to pick values for each hyperparameter.

5.2.4. Attack classification results used to select the ML approach for the first stage of our investigation

Table 8 displays each model's performance after performing GridSearchCV; Table 9 shows each model's performance after completing RandomizedSearchCV; and Table 10 and Fig. 11 displays the accuracy scores of the ML algorithms used to detect intrusions after feature selection.

Explanation of the Enhanced Random Forests hyperparameters:

- **n_estimators:** The number of decision trees in the forest. A higher number can lead to a more robust model but may require more computational resources.
- **max_depth:** The maximum depth of each decision tree. Setting it to None allows the trees to expand until they contain *min_samples_split* samples or fewer in each leaf.
- **min_samples_split:** The minimum number of samples required to split an internal node. Setting it to 2 means that a node must have at least 2 samples to be divided further.
- **min_samples_leaf:** The minimum number of samples required to be in a leaf node. This parameter can prevent the trees from growing too deep and overfitting.
- **max_features:** The number of features to consider when looking for the best split. 'auto' means it will consider all features.

These hyperparameters are commonly used as a starting point for an Enhanced Random Forest. The optimized values are identified using the aforementioned RandomizedSearchCV and GridSearchCV approaches and a brute force investigation.

Fig. 12 plots the AUC - ROC Curve for the five ML models we have trained. Based on the curve, it can be deduced that all the methods, including the Enhanced Random Forest classifier, outperform NBC. An additional observation on the AUC-ROC curves for our machine learning models is that the curves converge towards the plot's upper left corner. This visual clustering of curves directly results from the high classification accuracy achieved by all models, as reflected in the

Table 8
Grid search cross-validation results.

ML model	Best score	Best parameters
Support Vector Machine	0.994345	['C': 20, 'kernel': 'linear']
K-Nearest Neighbors	0.993934	['n_neighbours': 1]
Decision Tree	0.989356	['max_depth': 6]
Random Forest	0.999002	['n_estimators': 17]
Enhanced Random Forest	0.999002	['n_estimators': 17, 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'auto']
Naive Bayes	0.986655	[]

Table 9
Randomized search cross-validation results.

ML model	Best score	Best parameters
Support Vector Machine	0.994345	['C': 20, 'kernel': 'linear']
K-Nearest Neighbors	0.997422	['n_neighbours': 1]
Decision Tree	0.993934	['max_depth': 6]
Random Forest	0.998919	['n_estimators': 20]
Enhanced Random Forest	0.998919	['n_estimators': 20, 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'auto']
Naive Bayes	0.986655	[]

Table 10
Accuracy metrics of ML algorithms with feature selection.

Machine learning algorithms	Training sample size		
	(70%)	(60%)	(50%)
Enhanced Random Forest	99.96	99.98	99.98
Support Vector Machine	83.57	83.91	84.25
K-Nearest Neighbors	98.89	98.90	98.83
Decision Tree	99.63	99.97	99.97
Naive Bayes	98.06	98.02	98.10

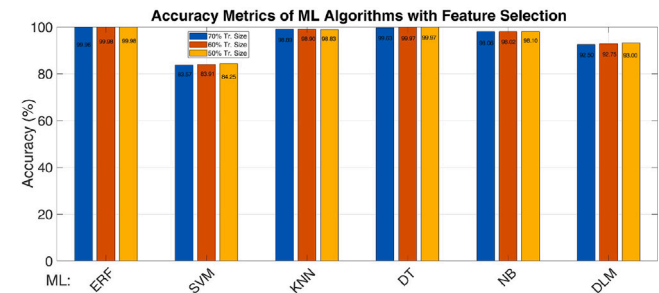


Fig. 11. ML algorithms accuracy.

precision, recall, and F1-score metrics reported. Each model demonstrated exceptional performance, with scores nearing the ideal value of 1, which leads to AUC values approaching unity except for the NBC. Given the robustness of the models and the consequent minimal variation in their performance metrics, the ROC curves are understandably indistinguishable. Such an occurrence is not uncommon in scenarios where classifiers achieve superior performance levels, rendering their true positive and false positive rates highly similar.

The intrusion detection model needs to predict the five output labels as “normal” :0, “MITM” :1, “DDoS” :2, “BFA” :3, and “NMAP” :4. The classification report function builds a visualizer showing the main classification metrics for the custom target names and above-inferred labels listed in Table 11. Thus, Table 11 shows the detailed classification performances of the ML models for different classes.

Fig. 13 is the partial visualization of a decision tree from an Enhanced Random Forest formed from our training dataset.

Note that for our experiments, we run the ML approaches using our generated dataset that is created using emulation. We cross-checked the results with other datasets such as CSE-CIC-IDS2018 [85] and NSL-KDD [86], and our results are very similar to the results given by the other datasets (the difference was in terms of 0,1%).

Table 11
Classification performances of ML models.

ML model	Traffic class	Precision	Recall	f1-score
Enhanced Random Forest	Normal	1.00	1.00	1.00
	MitM	1.00	1.00	1.00
	DDoS	1.00	1.00	1.00
	Brute Force Authentication	1.00	0.94	0.97
	NMAP	0.99	1.00	0.99
Support Vector Machines	Normal	0.99	0.98	0.99
	MitM	0.43	1.00	0.60
	DDoS	0.83	0.54	0.65
	Brute Force Authentication	0.91	0.94	0.93
	NMAP	0.51	0.79	0.62
K-Nearest Neighbors	Normal	1.00	0.99	0.99
	MitM	0.89	0.94	0.99
	DDoS	0.98	0.99	0.99
	Brute Force Authentication	1.00	0.88	0.94
	NMAP	0.98	0.97	0.99
Decision Tree	Normal	1.00	1.00	1.00
	MitM	1.00	0.97	0.98
	DDoS	0.99	1.00	0.99
	Brute Force Authentication	0.97	0.94	0.95
	NMAP	0.98	1.00	0.99
Naive Bayes	Normal	0.97	0.98	0.98
	MitM	0.43	0.31	0.36
	DDoS	1.00	0.99	0.99
	Brute Force Authentication	1.00	0.88	0.94
	NMAP	0.98	0.99	0.98

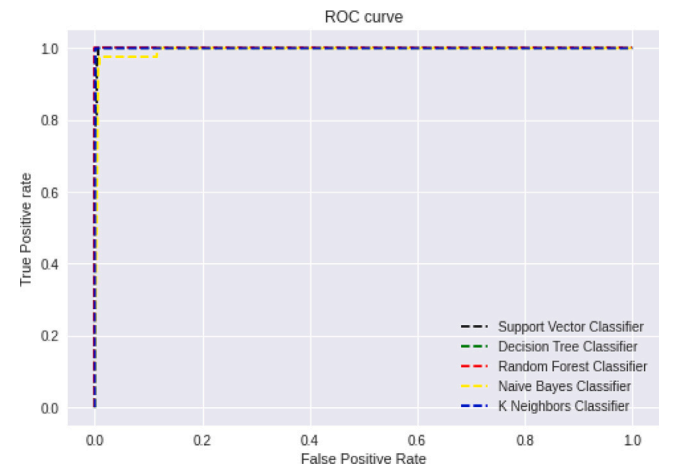


Fig. 12. AUC - ROC curve.

Table 13

Resource consumption of the ML models.

Models	Power consumption (mW)	CPU usage (%)	Memory usage (%)
Enhanced Random Forest	15.5	1	5
Support Vector Machine	11.1	1	5
K-Nearest Neighbours	15.6	1	5
Decision Tree	16.6	1	5
Naive Bayes	17.7	1	5
DL model	65	3	6

Table 14

Execution time and size of various models.

ML models	Execution time (s)	Model size (MB)
Enhanced Random Forest	1.06	1.369
Support Vector Machine	1.37	0.849
K-Nearest Neighbours	1.25	2.44
Decision Tree	0.54	0.01
Naive Bayes	0.86	0.003
DL model	0.94	0.008

Atop tool. The measurements obtained from these experiments were then documented on the Raspberry Pi, and the results are presented in Table 13 and Fig. 14. The table shown in this paper, specifically Table 14 and Fig. 15, provides information regarding the execution time and size of the machine learning (ML) models that have been loaded onto the Raspberry Pi 3b+ device.

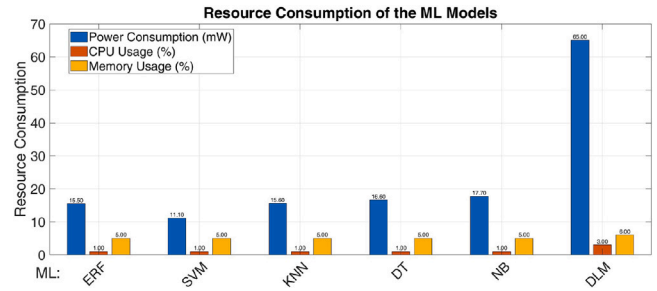
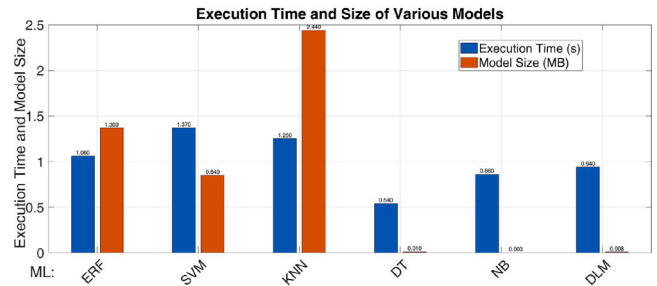
The power consumption of a Deep Learning (DL) model consisting of five hidden layers was generated in order to facilitate a comparative analysis with other Machine Learning (ML) models. The power consumption comparison between the utilized machine learning (ML) models and the deep learning (DL) model is evident in Table 13.

5.2.7. Rationale behind selecting the enhanced random forest model

The choice of the Enhanced Random Forest model for intrusion detection in MIoT networks was driven by a balanced consideration of various factors, as highlighted below:

- **Power Consumption and Execution Time:** While the Enhanced Random Forest model has a marginally higher power consumption (15.5 mW) and execution time (1.06 s), its high classification accuracy justifies these trade-offs, especially in the context of MIoT networks where accuracy is critical.
- **CPU and Memory Usage:** The model's CPU and memory usage stand at 1% and 5%, respectively, indicating its moderate resource requirements and making it suitable for resource-constrained environments in MIoT networks.
- **Model Size:** The Enhanced Random Forest model's size of 1.369 MB, while not the smallest, is compensated by its superior accuracy in classifying various traffic classes, as demonstrated in our results.
- **Classification Performance:** This model consistently achieves high precision, recall, and f1-scores across different traffic classes, thereby ensuring reliable detection of both normal and anomalous traffic in MIoT networks.
- **Balanced Approach:** Selecting the Enhanced Random Forest model represents a balanced approach, optimizing both resource utilization efficiency and effectiveness in accurately detecting a wide range of threats.

In conclusion, the Enhanced Random Forest model is identified as the 'golden model' for our study, offering an ideal blend of accuracy and practicality for intrusion detection in MIoT networks.

**Fig. 14.** ML models resource consumption.**Fig. 15.** ML models execution time and size.

5.2.8. Examining the results of the proposed approach with a well-known dataset

To thoroughly examine the performance and robustness of our proposed approach, extensive experiments were conducted using the "CICEV2023 DDoS Attack Dataset" [90]. This dataset is mainly chosen for its standardization and relevance in the domain of network security. Over 100 individual experiments were performed to ensure a comprehensive evaluation. The results from these extensive tests have been exceptionally consistent, demonstrating the effectiveness of our approach with variations in performance constrained to only $\pm 0.05\%$. This consistent performance across a significant number of trials validates our methodology's reliability and underscores its applicability in diverse real-world scenarios.

5.3. Evaluation of federated learning realization

In this section, we will examine the realization of the proposed approach shown in Section 4.3 and evaluate the FL approach as shown in Section 4.5. According to the findings in Section 5.2, the following components of the proposed approach are implemented in the realization of the emulation:

- *The GateWay IDS/MIoT GateWay (GW)* is implemented with a Raspberry PI 4 with routing and WiFi access point services enabled. The gateway was connected to the core network via cable..
- *The Anomaly Detector* is implemented using One-Class SVM, as shown in Section 5.2.5; it is the most appropriate method with the highest accuracy and identification results.
- *The Attack Classifier* is implemented using Enhanced Random Forests, utilizing Enhanced Random Forests, as described in Section 5.2.4; it is the method with the highest accuracy and classification outcomes.
- *The Threat Intelligence Federated Learning Server/The cloud server model* is implemented as another model that runs at an AWS cloud server. The FL implementation is shown in Section 4.5.

Additionally, in this section, we evaluate the FL approach regarding delay to calculate the deltas. Fig. 16 shows that the investigated approach of FL uses very little time to evaluate differences in the

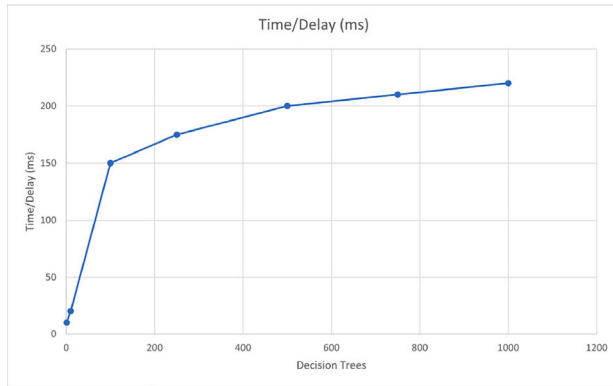


Fig. 16. Delta calculation execution time.

Enhanced Random Forests among trees starting from 1 to 1000 decision trees. Overall, our system is very light and fast, using Federated Learning. As shown in Section 2.2.6, the overall complexity is $O(n)$ (see Fig. 16).

5.3.1. Explanation of the federated learning approach with cosine similarity index

In our research, we have implemented the concept of federated learning in a novel approach by enhancing Random Forests, diverging from the conventional use of neural networks. This innovation is structured around the Depth-First Search (DFS) algorithm, which calculates the differences between old and new models within the federated learning framework. Specifically, the DFS algorithm assesses the Enhanced Random Forest models by subtracting the traversal result set of the old model from the new model. This process allows for an efficient and effective way to capture model updates, which are then communicated to other devices through a cloud-based model, focusing on the delta changes. To explain and interpret the proposed algorithm, we have utilized the Cosine Similarity Index.⁴ This metric has been instrumental in measuring the degree of change or similarity between model updates across different nodes in the federated network. Using the Cosine Similarity Index aids in ensuring the consistency and relevance of updates shared among devices, providing a quantitative measure to assess the alignment in learning patterns across the distributed system. Fig. 17 plots Cosine Similarity against Communication Rounds as part of an investigation into Federated Learning. Cosine Similarity is a measure that quantifies the similarity between two vectors, which, in the context of Federated Learning, typically represents the alignment of model parameters updated during training across different nodes. Communication rounds in Federated Learning refer to the iterative process where multiple nodes (such as mobile devices or distributed servers) each compute an update to the model based on their local data and then send it to a central server. The

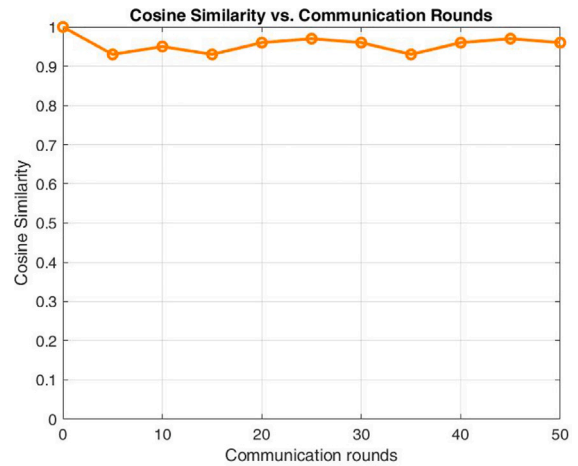


Fig. 17. The Cosine similarity index.

server aggregates these updates to improve the global model, which is then sent back to the nodes for further training. This process repeats for many rounds to refine the model progressively. In the graph, the Cosine Similarity starts at a value of 1, indicating that initially, all nodes have identical or maximally similar model parameters. As the number of communication rounds increases, the Cosine Similarity fluctuates slightly. Generally, it remains high, suggesting that the model parameters across different nodes remain similar even as local updates are applied. This can indicate that despite the diversity of local data and experiences, the nodes are learning consistently. The relatively stable Cosine Similarity across the communication rounds in the graph suggests that Federated Learning successfully integrates diverse local updates without significant divergence, which is a desirable outcome for such systems. This stability is key to ensuring that the global model benefits from the unique contributions of all participating nodes while maintaining a level of coherence and generalizability.

5.4. Time complexity of the proposed scheme

According to Sections 2.2.5 and 2.2.6, the computational complexity of our Federated Learning approach is aligned with that of Depth-First Search (DFS). The computational complexity of DFS is expressed as $O(V + E)$ for a network or tree. It simplifies to $O(N)$ for tree structures, owing to the $O(1)$ complexity of set operations [48]. In contrast, our Enhanced Random Forest (ERF) algorithm, which handles a dataset with 25 features, follows a complexity of $O(K \cdot N \cdot \log(N) \cdot F)$. This complexity further simplifies to $O(N \cdot \log(N))$ when considering our dataset comprising 19,208 training instances, as detailed in Table 7. Therefore, the overall system complexity, which combines the intricacies of both DFS and ERF in the Federated Learning framework, is collectively represented as $O(n) + O(N \cdot \log(N))$, effectively encompassing the efficiencies of both data structure traversal and ensemble learning techniques.

6. Conclusions and future work

This research presents a novel strategy with three stages for detecting intrusions on medical Internet of Things (IoT) devices at the sink-hole, utilizing a green machine learning methodology. In the first stage, the proposed method employed supervised learning machine learning classification techniques. These techniques were effectively trained and implemented on a Raspberry Pi sinkhole functioning as a gateway for

⁴ The Cosine Similarity Index is a measure used to determine the similarity between two vectors in a multi-dimensional space. It is calculated by finding the cosine of the angle between these vectors. In federated learning, this index helps compare model updates from different nodes by assessing the cosine similarity of these updates. A high value indicates similar learning patterns across nodes, while a low-value points to varied learning or data distributions. This understanding is essential for consistent and effective learning in a federated system. The formula for the Cosine Similarity Index between two vectors A and B is given by: $\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$. In this formula, $A \cdot B$ represents the dot product of vectors A and B , while $\|A\|$ and $\|B\|$ denote the magnitudes (or norms) of vectors A and B , respectively.

the Internet of Things (IoT) devices. The sinkhole in our proposed architectural design has identified instances of Man-in-the-Middle (MitM), Distributed Denial of Service (DDoS), Brute Force Authentication, and NMAP attacks. A network emulation was conducted using the KALI penetration testing software to generate a dataset. Initially stored as a PCAP file, the recently generated dataset was converted into a CSV file format. This conversion involved utilizing a “CICFlowmeter” tool, which ensured that the resulting information contained appropriate columnar features. This dataset, suitable for employment in our machine learning methodologies, will be accessible to the public. The classifiers Support Vector Machines, Naive Bayes, K -Nearest Neighbors, Decision Tree, and Enhanced Random Forest were employed to classify known assaults in the evaluation. Having the Enhanced Random Forest achieves the best overall accuracy with 99.98%. Subsequently, in the second stage, the One-Class Support Vector Machine (SVM) algorithm was employed to detect unidentified attacks such as Eavesdropping, Physical Tampering, and Device Impersonation. This approach was demonstrated to be the most suitable methodology for this particular undertaking, as supported by existing scholarly works with an accuracy of 99.7%. The results obtained from our emulation demonstrate that all the strategies under investigation exhibit notable performance in terms of accuracy, precision, recall metrics, and the F1 score. Ultimately, utilizing the acquired outcomes, we have successfully proven that all the examined machine learning methodologies can promptly and precisely identify all forms of attacks while exhibiting reduced utilization of memory, CPU, and power compared to alternative deep learning models. Furthermore, our study demonstrated that the Enhanced Random Forest algorithm exhibits superior accuracy and is considered one of the most efficient machine learning techniques regarding energy, CPU, and memory utilization compared to other approaches. Therefore, the methodology mentioned above will be implemented in the cloud-based machine learning model for training sinkholes/gateways in the MIIoT network. This training will incorporate the most recent updates of nodes and trees in the Enhanced Random Forest algorithm to enhance the identification of new attacks by utilizing Federated Learning techniques. As demonstrated, implementing Federated Learning results in minimal execution time for model updates and the transmission of differences, as measured in time. Finally, we form a complete, secure ERP system with the solutions provided.

In subsequent research, we intend to augment our dataset by incorporating additional attack types. We aim to assess the overall effectiveness of the cyber threat intelligence system in generating precise models for recently identified attacks while minimizing the time delay. Additionally, there are intentions to broaden the scope of the intrusion detection perimeter to encompass cloud servers and data storage units specifically designed for the IoT within the context of the Internet of Military Things. In addition, we will extend the existing system model under investigation to incorporate several designs for managing the Internet of Things. Also, for future work, it is essential to build upon the findings and implications of this study by exploring several critical areas. Firstly, investigating the scalability of the proposed Intrusion Detection System (IDS) across diverse and larger MIIoT networks is crucial to gaining deeper insights into its adaptability and robustness. Secondly, future work should consider conducting real-world field tests to evaluate the practical applicability and effectiveness of the IDS in various healthcare settings, which would be immensely beneficial. These areas of future work aim to significantly expand the scope and impact of the research, thereby contributing to the development of more secure and efficient MIIoT networks.

CRedit authorship contribution statement

Iacovos Ioannou: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization,

Writing – original draft, Writing – review & editing. **Prabagarane Nagaradjane:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Pelin Angin:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Palaniappan Balasubramanian:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Karthick Jeyagopal Kavitha:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Palani Murugan:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Vasos Vassiliou:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Dr Iacovos Ioannou reports financial support was provided by CYENS. Dr Iacovos Ioannou reports a relationship with CYENS Centre of Excellence Ltd that includes: employment.

Data availability

Data will be made available on request.

Acknowledgments

This research is part of a project that has received funding from the European Union's Horizon 2020 research and innovation program under grant agreement N° 739578 and the government of the Republic of Cyprus through the Directorate General for European Programmes, Coordination, and Development. The research has also been supported by a research grant from Middle East Technical University's Scientific Research Projects office under grant number GAP-312-2020-10297.

Funding

The authors received no specific funding for this work.

Consent for publication

All authors have approved the manuscript and agree with its submission to the relevant journal.

Appendix A. Glossary of terms and abbreviations

In this section, under the appendix, we provide the table of abbreviations used in the paper. [Table A.15](#) provides the abbreviation and description near it.

Table A.15

Abbreviations and descriptions.

Abbrev.	Description
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks
ANN	Artificial Neural Network
API	Application Programming Interface
AUC	Area Under Curve
CA	Certificate Authority
CHS	Connected Healthcare System
CNN	Convolutional Neural Networks
DAE	Deep Auto Encoder
DDoS	Distributed Denial of Service
DFNN	Deep Feed Forward Neural Network
DFS	Depth-First Search
DL	Deep Learning
DNN	Deep Neural Networks
DoS	Denial of Service
DT	Decision Trees
EMQX	EMQ X MQTT Broker
EOS-ELM	Ensemble of Online Sequential Extreme Learning Machine
ERF	Enhanced Random Forests
ESFCM	Extreme Learning Machine (ELM)-based Semi-supervised Fuzzy C-means (ESFCM)
FL	Federated Learning
FN	False Negatives
FNR	False Negative Rate
FP	False Positives
FPR	False Positive Rate
GEMLIDS	Green Effective Machine Learning Intrusion Detection System
GW	Gateway
GWO	Grey Wolf Optimization
HEKA	IDS name
HIDS	Host-based IDS
HTML	Hyper Text Markup Language
HTTP	HyperText Transfer Protocol
ICMP	Internet Control Message Protocol
ICS	Industrial Control Systems
IDS	Intrusion Detection Systems
IICS	Integrated ICS
IIoT	Industrial Internet of Things
IoMT	Internet of Medical Things
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LAN	Local Area Network
LDA	Linear Discriminant Analysis
LSTM	Long Short-Term Memory
MIOT	Medical IoT
MIoT	Medical Internet of Things
MitM	Man-in-the-Middle
ML	Machine Learning
MPC	Multi-Party Computation
MQTT	Message Queuing Telemetry Transport
MSGG	Message
NB	Naive Bayes
N-gram	A Sequence of N Words
NIDS	Network Intrusion Detection System
NMAP	Network Mapper
P	Precision
PCA	Principal Component Analysis
PCAP	Packet Captures
PMD	Patient Monitoring Devices
Probe	Prospective Randomized Open Blinded End-Point
R	Recall
R2L	Remote to User
REST	Representational State Transfer
RF	Random Forest
RNN	Random Neural Networks
ROC	Receiver Operator Characteristic
SCTP	Stream Control Transmission Protocol
SMOTE	Synthetic Minority Oversampling Technique
SOAP	Simple Object Access Protocol
SYM	Support Vector Machine

(continued on next page)

Table A.15 (continued).

Abbrev.	Description
SVMs	Support Vector Machines
SYN Flooding	Synchronization Flooding
TCP	Transmission Control Protocol
TDTC	Two-tier Classification Model
TN	True Negatives
TP	True Positives
TPR	True Positive Rate
U2R	User to Root Attacks
UDP	User Datagram Protocol
UNSW-NB15	University of New South Wales-NB15 dataset
WPAN	Wireless Personal Area Network
XGBoost	Extreme Gradient Boosting
YANG	Yet Another Next Generation

References

- [1] S. Gao, G. Thamilarasu, Machine-learning classifiers for security in connected medical devices, in: 2017 26th International Conference on Computer Communication and Networks, ICCCN, 2017, pp. 1–5, <http://dx.doi.org/10.1109/ICCCN.2017.8038507>.
- [2] D. He, Q. Qiao, Y. Gao, J. Zheng, S. Chan, J. Li, N. Guizani, Intrusion detection based on stacked autoencoder for connected healthcare systems, *IEEE Netw.* 33 (6) (2019) 64–69, <http://dx.doi.org/10.1109/MNET.001.1900105>.
- [3] A.I. Newaz, A.K. Sikder, L. Babun, A.S. Uluagac, HEKA: A novel intrusion detection system for attacks to personal medical devices, in: 2020 IEEE Conference on Communications and Network Security, CNS, 2020, pp. 1–9, <http://dx.doi.org/10.1109/CNS48642.2020.9162311>.
- [4] A. Odesile, G. Thamilarasu, Distributed intrusion detection using mobile agents in wireless body area networks, in: 2017 Seventh International Conference on Emerging Security Technologies, EST, 2017, pp. 144–149, <http://dx.doi.org/10.1109/EST.2017.8090414>.
- [5] S.P. R.M., P.K.R. Maddikunta, P. M., S. Koppu, T.R. Gadekallu, C.L. Chowdhary, M. Alazab, An effective feature engineering for DNN using hybrid PCA-GWO for intrusion detection in IoMT architecture, *Comput. Commun.* 160 (2020) 139–149, <http://dx.doi.org/10.1016/j.comcom.2020.05.048>, URL <https://www.sciencedirect.com/science/article/pii/S014036642030298X>.
- [6] P. Kumar, G.P. Gupta, R. Tripathi, An ensemble learning and fog-cloud architecture-driven cyber-attack detection framework for IoMT networks, *Comput. Commun.* 166 (2021) 110–124, <http://dx.doi.org/10.1016/j.comcom.2020.12.003>, URL <https://www.sciencedirect.com/science/article/pii/S0140366420320090>.
- [7] S.-R.J.-H.-P. Jae-Dong-Lee Hyo-Soung-Cha, M-IDM: A multi-classification based intrusion detection model in healthcare IoT, *Comput. Mater. Contin.* 67 (2) (2021) 1537–1553, <http://dx.doi.org/10.32604/cmc.2021.014774>, URL <http://www.techscience.com/cmc/v67n2/41342>.
- [8] A.A. Hady, A. Ghubaish, T. Salman, D. Unal, R. Jain, Intrusion detection system for healthcare systems using medical and network data: A comparison study, *IEEE Access* 8 (2020) 106576–106584, <http://dx.doi.org/10.1109/ACCESS.2020.3000421>.
- [9] I. Alrashdi, A. Alqazzaz, R. Alharthi, E. Aloufi, M.A. Zohdy, H. Ming, FBAD: Fog-based attack detection for IoT healthcare in smart cities, in: 2019 IEEE 10th Annual Ubiquitous Computing, Electronics Mobile Communication Conference, UEMCON, 2019, pp. 0515–0522, <http://dx.doi.org/10.1109/UEMCON47517.2019.8992963>.
- [10] S. Khan, A. Akhuzada, A hybrid DL-driven intelligent SDN-enabled malware detection framework for internet of medical things (IoMT), *Comput. Commun.* 170 (2021) 209–216, <http://dx.doi.org/10.1016/j.comcom.2021.01.013>, URL <https://www.sciencedirect.com/science/article/pii/S0140366421000347>.
- [11] A. Raza, K.P. Tran, L. Koehl, S. Li, AnoFed: Adaptive anomaly detection for digital health using transformer-based federated learning and support vector data description, *Eng. Appl. Artif. Intell.* 121 (May 2022) (2023) 106051, <http://dx.doi.org/10.1016/j.engappai.2023.106051>.
- [12] P. Kasinathan, C. Pastrone, M.A. Spirito, M. Vinkovits, Denial-of-service detection in 6LoWPAN based internet of things, in: 2013 IEEE 9th International Conference on Wireless and Mobile Computing, Networking and Communications, WiMob, IEEE, 2013, pp. 600–607.
- [13] A.-H. Muna, N. Moustafa, E. Sitnikova, Identification of malicious activities in industrial internet of things based on deep learning models, *J. Inf. Secur. Appl.* 41 (2018) 1–11.
- [14] D. Oh, D. Kim, W. Ro, A malicious pattern detection engine for embedded security systems in the internet of things, *Sensors* 14 (12) (2014) 24188–24211.
- [15] S. Evmoros, G. Vlachodimitropoulos, N. Bakalos, E. Gelenbe, Neural network architectures for the detection of SYN flood attacks in IoT systems, in: Proceedings of the 13th ACM International Conference on Pervasive Technologies Related to Assistive Environments, PETRA'20, Association for Computing Machinery, New York, NY, USA, 2020, <http://dx.doi.org/10.1145/3389189.3398000>.

- [16] Y.N. Soe, Y. Feng, P.I. Santosa, R. Hartanto, K. Sakurai, Machine learning-based IoT-botnet attack detection with sequential architecture, *Sensors* 20 (16) (2020) 4372, <http://dx.doi.org/10.3390/s20164372>.
- [17] S. Rathore, J.H. Park, Semi-supervised learning based distributed attack detection framework for IoT, *Appl. Soft Comput.* 72 (2018) 79–89, <http://dx.doi.org/10.1016/j.asoc.2018.05.049>, URL <http://www.sciencedirect.com/science/article/pii/S1568494618303508>.
- [18] E.J. Cho, J.H. Kim, C.S. Hong, Attack model and detection scheme for botnet on 6lowpan, in: *Asia-Pacific Network Operations and Management Symposium*, Springer, 2009, pp. 515–518.
- [19] N.K. Thanigaivelan, E. Nigusie, R.K. Kanth, S. Virtanen, J. Isoaho, Distributed internal anomaly detection system for internet-of-things, in: 2016 13th IEEE Annual Consumer Communications & Networking Conference, CCNC, IEEE, 2016, pp. 319–320.
- [20] D.H. Summerville, K.M. Zach, Y. Chen, Ultra-lightweight deep packet anomaly detection for internet of things devices, in: 2015 IEEE 34th International Performance Computing and Communications Conference, IPCCC, IEEE, 2015, pp. 1–8.
- [21] T.-H. Lee, C.-H. Wen, L.-H. Chang, H.-S. Chiang, M.-C. Hsieh, A lightweight intrusion detection scheme based on energy consumption analysis in 6Low-PAN, in: *Advanced Technologies, Embedded and Multimedia for Human-Centric Computing*, Springer, 2014, pp. 1205–1213.
- [22] P. Pongle, G. Chavan, Real time intrusion and wormhole attack detection in internet of things, *Int. J. Comput. Appl.* 121 (9) (2015).
- [23] S. Zhao, W. Li, T. Zia, A.Y. Zomaya, A dimension reduction model and classifier for anomaly-based intrusion detection in internet of things, in: 2017 IEEE 15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress, DASC/PiCom/DataCom/CyberSciTech, IEEE, 2017, pp. 836–843.
- [24] H.H. Pajouh, R. Javidan, R. Khayami, D. Ali, K.-K.R. Choo, A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks, *IEEE Trans. Emerg. Top. Comput.* (2016).
- [25] M.J. Idrissi, H. Alami, A. El Mahdaoui, A. El Mekki, S. Oualil, Z. Yartaoui, I. Berrada, Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems, *Expert Syst. Appl.* 234 (June) (2023) <http://dx.doi.org/10.1016/j.eswa.2023.121000>.
- [26] X. Wang, Y. Wang, Z. Javaheri, L. Almutairi, N. Moghadamnejad, O.S. Younes, Federated deep learning for anomaly detection in the internet of things, *Comput. Electr. Eng.* 108 (March) (2023) 108651, <http://dx.doi.org/10.1016/j.compeleceng.2023.108651>.
- [27] B. Weinger, J. Kim, A. Sim, M. Nakashima, N. Moustafa, K.J. Wu, Enhancing IoT anomaly detection performance for federated learning, *Digit. Commun. Netw.* 8 (3) (2022) 314–323, <http://dx.doi.org/10.1016/j.dcan.2022.02.007>.
- [28] H. Alaiz-Moreton, J. Avelaira-Mata, J. Ondicol-Garcia, A.L. Muñoz-Castañeda, I. García, C. Benavides, Multiclass classification procedure for detecting attacks on MQTT-IoT protocol, *Complexity* 2019 (2019) 1–12.
- [29] C. Wang, Y. Sun, S. Lv, C. Wang, H. Liu, B. Wang, Intrusion detection system based on one-class support vector machine and Gaussian mixture model, *Electronics* 12 (4) (2023) 930.
- [30] E. Borgia, The internet of things vision: Key features, applications and open issues, *Comput. Commun.* 54 (2014) 1–31.
- [31] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of things: A survey on enabling technologies, protocols, and applications, *IEEE Commun. Surv. Tutorials* 17 (4) (2015) 2347–2376.
- [32] R.T. Fielding, REST: Architectural Styles and the Design of Network-Based Software Architectures (Doctoral Dissertation), University of California, 2000.
- [33] P. Charles, L. Rabejac, Secure communications and man-in-the-middle, in: *International Workshop on Security Protocols*, Springer, 2002, pp. 31–37.
- [34] J. Mirkovic, P. Reiher, F. Shepherd, Modeling and defending against DDoS attacks, *Proc. IEEE* 92 (2) (2004) 317–331.
- [35] M. Nitta, A. Hirano, M. Miura, Efficient brute-force attack search algorithms, in: *Proceedings of the 2009 ACM Workshop on Cloud Computing Security*, ACM, 2009, pp. 13–18.
- [36] S. Raghuvaran, A. Goel, R.T. Rajan, C.V. Hota, Real-time detection of NMAP scans, in: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS*, IEEE, 2016, pp. 2615–2620.
- [37] V. Jakkula, Tutorial on support vector machine (svm), *Sch. EECS Washington State Univ.* 37 (2.5) (2006) 3.
- [38] A.D. Shieh, D.F. Kamm, Ensembles of one class support vector machines, in: *International Workshop on Multiple Classifier Systems*, Springer, 2009, pp. 181–190.
- [39] S. Dreiseitl, M. Osl, C. Scheibböck, M. Binder, Outlier detection with one-class SVMs: an application to melanoma prognosis, in: *AMIA Annual Symposium Proceedings*, Vol. 2010, American Medical Informatics Association, 2010, p. 172.
- [40] N. Shahid, I.H. Naqvi, S.B. Qaisar, One-class support vector machines: analysis of outlier detection for wireless sensor networks in harsh environments, *Artif. Intell. Rev.* 43 (4) (2015) 515–563.
- [41] C. Lu, J. Huang, L. Huang, Detecting urban anomalies using factor analysis and one class support vector machine, *Comput. J.* 66 (2) (2023) 373–383.
- [42] O. Kramer, O. Kramer, K-nearest neighbors, in: *Dimensionality Reduction with Unsupervised Nearest Neighbors*, Springer, 2013, pp. 13–23.
- [43] A.J. Myles, R.N. Feudale, Y. Liu, N.A. Woody, S.D. Brown, An introduction to decision tree modeling, *J. Chemometr.: J. Chemometr. Soc.* 18 (6) (2004) 275–285.
- [44] G.I. Webb, E. Keogh, R. Miikkulainen, Naïve Bayes., *Encyclopedia Mach. Learn.* 15 (1) (2010) 713–714.
- [45] A. Cutler, D.R. Cutler, J.R. Stevens, Random forests, in: *Ensemble Machine Learning: Methods and Applications*, Springer, 2012, pp. 157–175.
- [46] S. Bernard, L. Heutte, S. Adam, On the selection of decision trees in random forests, in: 2009 International Joint Conference on Neural Networks, IEEE, 2009, pp. 302–307.
- [47] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32, <http://dx.doi.org/10.1023/A:1010933404324>.
- [48] I. Chivers, J. Sleightholme, I. Chivers, J. Sleightholme, An introduction to algorithms and the big O notation, in: *Introduction to Programming with Fortran: With Coverage of Fortran 90, 95, 2003, 2008 and 77*, Springer, 2015, pp. 359–364.
- [49] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [50] Z. Chai, C. Zhao, Enhanced random forest with concurrent analysis of static and dynamic nodes for industrial fault classification, *IEEE Trans. Ind. Inform.* 16 (1) (2019) 54–66.
- [51] A. Liaw, M. Wiener, Classification and regression by randomforest, *R News* 2 (3) (2002) 18–22.
- [52] C. Chen, A. Liaw, L. Breiman, Random forests for imbalanced data, in: *Proceedings of the International Conference on Machine Learning, ICML*, 2010.
- [53] C. Chen, A. Liaw, L. Breiman, Using random forest to learn imbalanced data, in: *Proceedings of the International Conference on Machine Learning, ICML*, 2004.
- [54] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, in: *Proceedings of the International Conference on Machine Learning, ICML*, 2012.
- [55] D. Chicco, L. Oneto, An enhanced random forests approach to predict heart failure from small imbalanced gene expression data, *IEEE/ACM Trans. Comput. Biol. Bioinform.* 18 (6) (2020) 2759–2765.
- [56] Y. Liu, J. Chen, Z. Su, Z. Luo, N. Luo, L. Liu, K. Zhang, Robust head pose estimation using Dirichlet-tree distribution enhanced random forests, *Neurocomputing* 173 (2016) 42–53.
- [57] D. Amarutunga, J. Cabrera, Y.S. Lee, Enriched random forests, *Bioinformatics* 24 (18) (2008) 2010–2014.
- [58] X. Liu, T. Fu, Z. Pan, D. Liu, W. Hu, J. Liu, K. Zhang, Automated layer segmentation of retinal optical coherence tomography images using a deep feature enhanced structured random forests classifier, *IEEE J. Biomed. Health Inform.* 23 (4) (2018) 1404–1416.
- [59] Z. Yang, M. Chen, K.-K. Wong, H.V. Poor, S. Cui, Federated learning for 6G: Applications, challenges, and opportunities, *Engineering* 8 (2022) 33–41, <http://dx.doi.org/10.1016/j.eng.2021.12.002>, URL <https://www.sciencedirect.com/science/article/pii/S2095809921005245>.
- [60] J. Konečný, B. McMahan, D. Ramage, Federated optimization: Distributed optimization beyond the datacenter, 2015, arXiv preprint arXiv:1511.03575.
- [61] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A.N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., Advances and open problems in federated learning, *Found. Trends® Mach. Learn.* 14 (1–2) (2021) 1–210.
- [62] E. Diao, J. Ding, V. Tarokh, HeteroFL: Computation and communication efficient federated learning for heterogeneous clients, 2020, arXiv preprint arXiv:2010.01264.
- [63] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Artificial Intelligence and Statistics, PMLR*, 2017, pp. 1273–1282.
- [64] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H.B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.
- [65] H. Li, J. Konečný, P. Richtárik, A. Sahu, C. Zhao, Federated quantization for communication-efficient collaborative learning, 2020, arXiv preprint arXiv:2007.07404.
- [66] R. Hu, Y. Guo, H. Li, Q. Pei, Y. Gong, Personalized federated learning with differential privacy, *IEEE Internet Things J.* 7 (10) (2020) 9530–9539.
- [67] J.R. Vacca, *Computer and Information Security Handbook*, Newnes, 2012.
- [68] S. Raza, L. Wallgren, T. Voigt, SVELTE: Real-time intrusion detection in the internet of things, *Ad Hoc Netw.* 11 (8) (2013) 2661–2674.
- [69] C. Cervantes, D. Poplade, M. Nogueira, A. Santos, Detection of sinkhole attacks for supporting secure routing on 6LoWPAN for internet of things, in: 2015 IFIP/IEEE International Symposium on Integrated Network Management, IM, IEEE, 2015, pp. 606–611.
- [70] L. Wallgren, S. Raza, T. Voigt, Routing attacks and countermeasures in the RPL-based internet of things, *Int. J. Distrib. Sens. Netw.* 9 (8) (2013) 794326.
- [71] M.H. Ali, M.M. Jaber, S.K. Abd, A. Rehman, M.J. Awan, R. Damaševićius, S.A. Bahaj, Threat analysis and distributed denial of service (DDoS) attack recognition in the internet of things (IoT), *Electronics* 11 (3) (2022) 494.

- [72] B. Bhushan, G. Sahoo, A.K. Rai, Man-in-the-middle attack in wireless and computer networking—A review, in: 2017 3rd International Conference on Advances in Computing, Communication & Automation, ICACCA Fall, IEEE, 2017, pp. 1–6.
- [73] M.M. Alani, Detection of reconnaissance attacks on IoT devices using deep neural networks, in: *Advances in Nature-Inspired Cyber Security and Resilience*, Springer, 2021, pp. 9–27.
- [74] X. Gong, Y. Chen, H. Huang, Y. Liao, S. Wang, Q. Wang, Coordinated backdoor attacks against federated learning with model-dependent triggers, *IEEE Netw.* 36 (1) (2022) 84–90.
- [75] N. Bouacida, P. Mohapatra, Vulnerabilities in federated learning, *IEEE Access* 9 (2021) 63229–63249.
- [76] V. Tolpegin, S. Truex, M.E. Gursoy, L. Liu, Data poisoning attacks against federated learning systems, in: *Computer Security—ESORICS 2020: 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14–18, 2020, Proceedings, Part I 25*, Springer, 2020, pp. 480–501.
- [77] D.G. Altman, Categorising continuous variables., *Br. J. Cancer* 64 (5) (1991) 975.
- [78] W.-Y. Loh, Improving the precision of classification trees, *Ann. Appl. Stat.* (2009) 1710–1737.
- [79] D. Huang, R. Li, H. Wang, Feature screening for ultrahigh dimensional categorical data with applications, *J. Bus. Econom. Statist.* 32 (2) (2014) 237–244.
- [80] G.E.A.P.A. Batista, R.C. Prati, M.C. Monard, A study of the behavior of several methods for balancing machine learning training data, *SIGKDD Explor. Newsl.* 6 (1) (2004) 20–29, <http://dx.doi.org/10.1145/1007730.1007735>, URL <https://doi.org/10.1145/1007730.1007735>.
- [81] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, *J. Artif. Int. Res.* 16 (1) (2002) 321–357.
- [82] J.D. Rodriguez, A. Perez, J.A. Lozano, Sensitivity analysis of k-fold cross validation in prediction error estimation, *IEEE Trans. Pattern Anal. Mach. Intell.* 32 (3) (2009) 569–575.
- [83] G.N. Ahmad, H. Fatima, S. Ullah, A.S. Saidi, et al., Efficient medical diagnosis of human heart diseases using machine learning techniques with and without GridSearchCV, *IEEE Access* 10 (2022) 80151–80173.
- [84] M. Vishnu, V.V. Rupak, S. Vedhapriya, M. Sangeetha, R. Manjuladevi, C. Sagana, Recurrent gastric cancer prediction using randomized search cv optimizer, in: 2023 International Conference on Computer Communication and Informatics, ICCCI, IEEE, 2023, pp. 1–5.
- [85] D. Ravikumar, Towards Enhancement of Machine Learning Techniques using CSE-CIC-IDS2018 Cybersecurity Dataset, Rochester Institute of Technology, 2021.
- [86] P. Bisen, A. Vishwakarma, et al., Machine learning based intrusion detection from wireless sensor network over NSL-KDD dataset, *IJRAR Int. J. Res. Anal. Rev. (IJRAR)* 7 (1) (2020) 683–688.
- [87] J.H. Anajemba, C. Iwendi, I. Razzak, J.A. Ansere, I.M. Okpalaoguchi, A counter-eavesdropping technique for optimized privacy of wireless industrial iot communications, *IEEE Trans. Ind. Inform.* 18 (9) (2022) 6445–6454.
- [88] P. Varga, S. Plosz, G. Soos, C. Hegedus, Security threats and issues in automation IoT, in: 2017 IEEE 13th International Workshop on Factory Communication Systems, WFCS, IEEE, 2017, pp. 1–6.
- [89] S.A. Chaudhry, K. Yahya, F. Al-Turjman, M.-H. Yang, A secure and reliable device access control scheme for IoT based sensor cloud systems, *IEEE Access* 8 (2020) 139244–139254.
- [90] Y. Kim, S. Hakak, A. Ghorbani, DDoS attack dataset (CICEV2023) against EV authentication in charging infrastructure, in: *Proceedings of the 20th International Conference on Privacy, Security, and Trust, PST2023, Copenhagen, Denmark, 2023*.